

Combinatorial Solving with Provably Correct Results

Lecture 2

Jakob Nordström

University of Copenhagen and Lund University

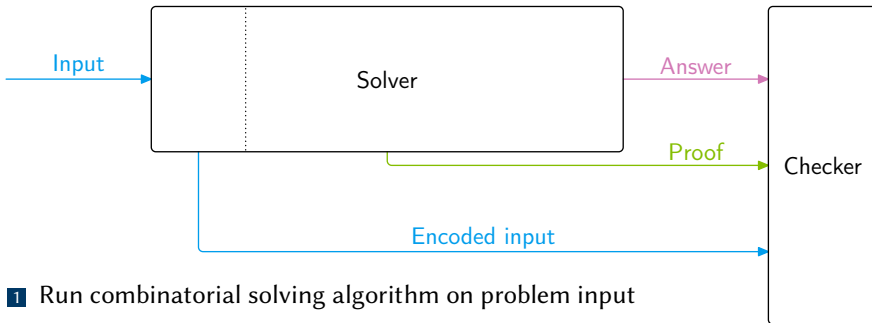
Topics in Algorithms and Complexity

University of Copenhagen

October 6, 2026

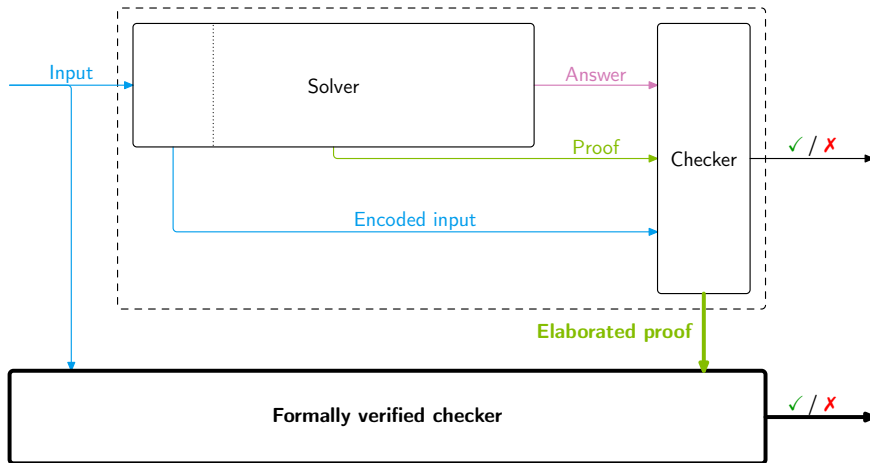


Recap of Proof Logging Workflow (1/2)



- 1 Run combinatorial solving algorithm on problem input
- 2 Get as output not only answer but also proof
- 3 Feed answer + proof to proof checker together with 0-1 ILP encoding of input

Recap of Proof Logging Workflow (2/2)



Implementation and Performance

Proof logging implementation

- Don't change solver
- Just add proof logging statements (plus some book-keeping)
- Proof generation should be very efficient

Performance goals for proof checking

Want linear(ish) scaling in terms of original **solver running time** for

- **proof size**
- **proof checking time**

VERIPB Proof Structure

1 Preamble

Specify

- VERIPB proof version
- # input constraints (optional)

VERIPB Proof Structure

1 Preamble

Specify

- VERIPB proof version
- # input constraints (optional)

2 Derivation section

The actual proof

Derivations of new constraints

Logging of solutions

3 Output section

List of constraints currently in database (optional)

Input to next stage (or for debugging)

4 Conclusions section

Specification of what was proven

- satisfiability / unsatisfiability
- optimality (or upper and lower bounds)
- enumeration of solutions
- other types of conclusions (future)

VERIPB Proof Structure: Syntax

pseudo-Boolean proof version 3.0

f $\langle M \rangle$;

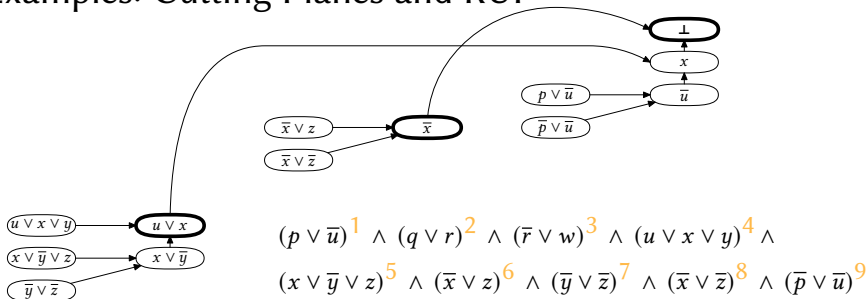
$\langle derivation part \rangle$

output $\langle output part \rangle$

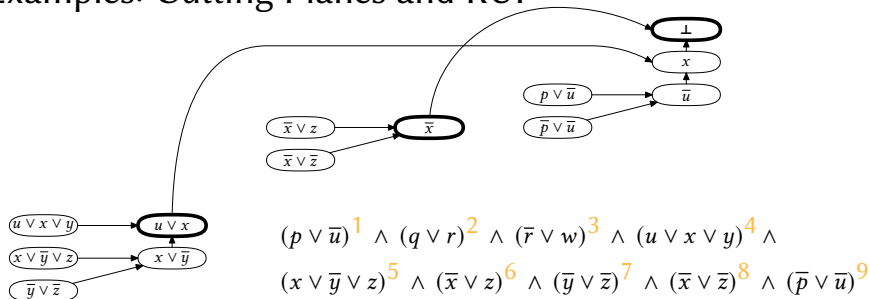
conclusion $\langle conclusion part \rangle$

end pseudo-Boolean proof ;

CDCL Proof Examples: Cutting Planes and RUP



CDCL Proof Examples: Cutting Planes and RUP



pol 7 5 + 2 d 4 + 2 d ;

pol 8 6 + 2 d ;

pol 9 1 + 2 d 10 + 2 d 11 + 2 d ;

$\rightsquigarrow$ Constraint 10 $\doteq u + x \geq 1$

$\rightsquigarrow$ Constraint 11 $\doteq \bar{x} \geq 1$

$\rightsquigarrow$ Constraint 12 $\doteq 0 \geq 1$ ⚡

VERIPB Proof Configuration

Core set $\mathcal{C}$

- Contains input formula at the start
- Input constraints can be modified (e.g., during preprocessing)
- But “equivalence” with input formula maintained

Derived set $\mathcal{D}$

- All constraints derived during search
- Also intermediate constraints used in proof logging [but not used by solver]
- Any satisfying assignment to $\mathcal{C}$ can be extended to $\mathcal{D}$

Objective $f = \sum_i w_i \ell_i + k$

- 0–1 linear function to minimise
- Or $f = 0$ for decision problem
- Keep track of best known bound; initialise to ∞

Reverse Unit Propagation (RUP) for Pseudo-Boolean Constraints

Disjunctive clause C is **RUP** w.r.t. F if

- assigning C to false
- then unit propagating on F until saturation
- leads to contradiction

How to generalize to pseudo-Boolean constraints? Can be falsified in many ways...

Reverse Unit Propagation (RUP) for Pseudo-Boolean Constraints

Disjunctive clause C is **RUP** w.r.t. F if

- ~~assigning C to false~~ adding negation $\neg C$
- then unit propagating on F until saturation
- leads to contradiction

How to generalize to pseudo-Boolean constraints? Can be falsified in many ways...

RUP for Pseudo-Boolean Constraints: Example

Is $D \doteq \overline{geq}_{11} + \overline{lt}_{11} \geq 1$ (i.e., $geq_{11} \Rightarrow \overline{lt}_{11}$) RUP with respect to $\{C_1, C_2\}$ below?

$$C_1 \doteq 11\overline{geq}_{11} + 8b_3 + 4b_2 + 2b_1 + b_0 \geq 11 \quad \text{i.e., } geq_{11} \Rightarrow (8b_3 + 4b_2 + 2b_1 + b_0 \geq 11)$$

$$C_2 \doteq 5\overline{lt}_{11} + 8\overline{b}_3 + 4\overline{b}_2 + 2\overline{b}_1 + \overline{b}_0 \geq 5 \quad \text{i.e., } lt_{11} \Rightarrow (8\overline{b}_3 + 4\overline{b}_2 + 2\overline{b}_1 + \overline{b}_0 \geq 5)$$

$$\neg D \doteq geq_{11} + lt_{11} \geq 2 \quad \text{i.e., } geq_{11} \wedge lt_{11}$$

$$\neg D \doteq \textcolor{green}{geq}_{11} + \textcolor{teal}{lt}_{11} \geq 2 \quad \text{slack} = 0$$

RUP for Pseudo-Boolean Constraints: Example

Is $D \doteq \overline{geq}_{11} + \overline{lt}_{11} \geq 1$ (i.e., $geq_{11} \Rightarrow \overline{lt}_{11}$) RUP with respect to $\{C_1, C_2\}$ below?

$$C_1 \doteq 11\overline{geq}_{11} + 8b_3 + 4b_2 + 2b_1 + b_0 \geq 11 \quad \text{slack} = 0$$

$$C_2 \doteq 5\overline{lt}_{11} + 8\overline{b}_3 + 4\overline{b}_2 + 2\overline{b}_1 + \overline{b}_0 \geq 5 \quad \text{slack} = 2$$

$$\neg D \doteq geq_{11} + lt_{11} \geq 2 \quad \text{slack} = 0$$

Unit propagation setting literals **true** or **false**:

$$geq_{11} \stackrel{\neg D}{=} 1, \quad lt_{11} \stackrel{\neg D}{=} 1, \quad b_3 \stackrel{C_1}{=} 1, \quad b_2 \stackrel{C_2}{=} 0$$

RUP for Pseudo-Boolean Constraints: Example

Is $D \doteq \overline{geq}_{11} + \overline{lt}_{11} \geq 1$ (i.e., $geq_{11} \Rightarrow \overline{lt}_{11}$) RUP with respect to $\{C_1, C_2\}$ below?

$$C_1 \doteq 11\overline{geq}_{11} + 8b_3 + 4b_2 + 2b_1 + b_0 \geq 11 \quad \text{slack} = 0$$

$$C_2 \doteq 5\overline{lt}_{11} + 8\overline{b}_3 + 4\overline{b}_2 + 2\overline{b}_1 + \overline{b}_0 \geq 5 \quad \text{slack} = -1$$

$$\neg D \doteq \textcolor{brown}{geq}_{11} + \textcolor{brown}{lt}_{11} \geq 2 \quad \text{slack} = 0$$

Unit propagation setting literals **true** or **false**:

$$geg_{11} \stackrel{\neg D}{=} 1, lt_{11} \stackrel{\neg D}{=} 1, b_3 \stackrel{C_1}{=} 1, b_2 \stackrel{C_2}{=} 0, b_1 \stackrel{C_1}{=} 1, b_0 \stackrel{C_1}{=} 1$$

RUP for Pseudo-Boolean Constraints: Example

Is $D \doteq \overline{geq}_{11} + \overline{lt}_{11} \geq 1$ (i.e., $geq_{11} \Rightarrow \overline{lt}_{11}$) RUP with respect to $\{C_1, C_2\}$ below?

$$C_1 \doteq 11\overline{geq}_{11} + 8b_3 + 4b_2 + 2b_1 + b_0 \geq 11 \quad \text{slack} = 0$$

$$C_2 \doteq 5\overline{lt}_{11} + 8\overline{b}_3 + 4\overline{b}_2 + 2\overline{b}_1 + \overline{b}_0 \geq 5 \quad \text{slack} = -1 \quad \textcolor{violet}{\text{⚡}}$$

$$\neg D \doteq geq_{11} + lt_{11} \geq 2 \quad \text{slack} = 0$$

Unit propagation setting literals **true** or **false**:

$$geq_{11} \stackrel{\neg D}{=} 1, \quad lt_{11} \stackrel{\neg D}{=} 1, \quad b_3 \stackrel{C_1}{=} 1, \quad \textcolor{violet}{b}_2 \stackrel{C_2}{=} 0, \quad b_1 \stackrel{C_1}{=} 1, \quad b_0 \stackrel{C_1}{=} 1, \quad \textcolor{violet}{\perp}^{C_2}$$

$$\neg D \doteq \textcolor{brown}{geq}_{11} + \textcolor{brown}{lt}_{11} \geq 2 \quad \text{slack} = 0$$

$$\neg D \doteq \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2 \quad \text{slack} = 1$$

$$\neg D \doteq \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2 \quad \text{slack} = 1$$

Strengthening Rules in Their (Almost) Full Formal Glory

Witness ω : substitution mapping variables to truth values or literals

Redundance-based strengthening (witness ω shows how to “patch assignment”)

Derive constraint C from $\mathcal{C} \cup \mathcal{D}$ if exists witness ω such that

$$\mathcal{C} \cup \mathcal{D} \cup \{\neg C\} \vdash (\mathcal{C} \cup \mathcal{D} \cup \{C\}) \upharpoonright_{\omega} \cup \{f \upharpoonright_{\omega} \leq f\} \cup \mathcal{O}(\vec{z} \upharpoonright_{\omega}, \vec{z})$$

Dominance-based strengthening (witness ω “drives down potential”)

Derive constraint C from $\mathcal{C} \cup \mathcal{D}$ if exists witness ω such that

$$\mathcal{C} \cup \mathcal{D} \cup \{\neg C\} \vdash \mathcal{C} \upharpoonright_{\omega} \cup \{f \upharpoonright_{\omega} \leq f\} \cup \mathcal{O}(\vec{z} \upharpoonright_{\omega}, \vec{z}) \cup \neg \mathcal{O}(\vec{z}, \vec{z} \upharpoonright_{\omega})$$

Syntax for Strengthening Rules

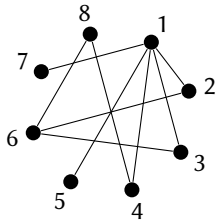
```
red ⟨Constraint C⟩ : ⟨var1⟩ -> ⟨val1⟩ . . . ⟨varN⟩ -> ⟨valN⟩ : subproof
    subproofs for proof goals
qed;
```

```
dom ⟨Constraint C⟩ : ⟨var1⟩ -> ⟨val1⟩ . . . ⟨varN⟩ -> ⟨valN⟩ : subproof
    subproofs for proof goals
qed;
```

Topics '26 15/79

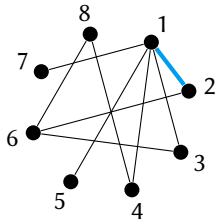
More Advanced Example of Redundance-Based Strengthening

Is the following graph 2-colourable (bipartite)?



More Advanced Example of Redundance-Based Strengthening

Is the following graph 2-colourable (bipartite)?

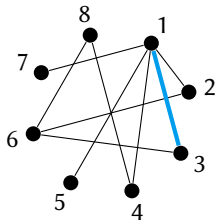


$$b_1 + b_2 \geq 1$$

$$\bar{b}_1 + \bar{b}_2 \geq 1$$

More Advanced Example of Redundance-Based Strengthening

Is the following graph 2-colourable (bipartite)?



$$b_1 + b_2 \geq 1$$

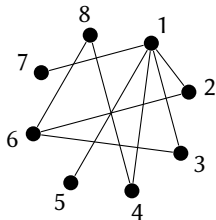
$$\bar{b}_1 + \bar{b}_2 \geq 1$$

$$b_1 + b_3 \geq 1$$

$$\bar{b}_1 + \bar{b}_3 \geq 1$$

More Advanced Example of Redundance-Based Strengthening

Is the following graph 2-colourable (bipartite)?



$$b_1 + b_2 \geq 1$$

$$\bar{b}_1 + \bar{b}_2 \geq 1$$

$$b_1 + b_3 \geq 1$$

$$\bar{b}_1 + \bar{b}_3 \geq 1$$

• • •

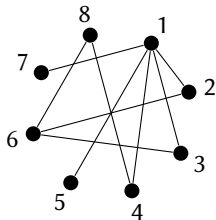
Without loss of generality node 1 is blue (otherwise we can swap the two colours)

Derive $b_1 \geq 1$ with redundance rule;
witness $\omega : b_i \mapsto \bar{b}_i$ for all i

$$F \doteq F \upharpoonright_\omega \quad \text{and} \quad \neg(b_1 \geq 1) = (b_1 \geq 1) \upharpoonright_\omega$$

More Advanced Example of Redundance-Based Strengthening

Is the following graph 2-colourable (bipartite)?



$$b_1 + b_2 \geq 1$$

$$\bar{b}_1 + \bar{b}_2 \geq 1$$

$$b_1 + b_3 \geq 1$$

$$\bar{b}_1 + \bar{b}_3 \geq 1$$

...

Without loss of generality node 1 is blue
(otherwise we can swap the two colours)

Derive $b_1 \geq 1$ with redundance rule;
witness $\omega : b_i \mapsto \bar{b}_i$ for all i

$$F \doteq F \upharpoonright_{\omega} \quad \text{and} \quad \neg(b_1 \geq 1) = (b_1 \geq 1) \upharpoonright_{\omega}$$

pseudo-Boolean proof version 3.0

f 18 ;

red 1 b1 >= 1 : b1 -> ~b1 b2 -> ~b2

↪ b3 -> ~b3 b4 -> ~b4 b5 -> ~b5

↪ b6 -> ~b6 b7 -> ~b7 b8 -> ~b8 ;

rup >= 1 ;

output NONE ;

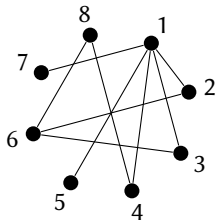
conclusion UNSAT ;

end pseudo-Boolean proof ;

■ Start the proof

More Advanced Example of Redundance-Based Strengthening

Is the following graph 2-colourable (bipartite)?



$$b_1 + b_2 \geq 1$$

$$\bar{b}_1 + \bar{b}_2 \geq 1$$

$$b_1 + b_3 \geq 1$$

$$\bar{b}_1 + \bar{b}_3 \geq 1$$

...

Without loss of generality node 1 is blue
(otherwise we can swap the two colours)

Derive $b_1 \geq 1$ with redundance rule;
witness $\omega : b_i \mapsto \bar{b}_i$ for all i

$$F \doteq F \upharpoonright_{\omega} \quad \text{and} \quad \neg(b_1 \geq 1) = (b_1 \geq 1) \upharpoonright_{\omega}$$

pseudo-Boolean proof version 3.0

f 18 ;

red 1 b1 >= 1 : b1 -> ~b1 b2 -> ~b2

↪ b3 -> ~b3 b4 -> ~b4 b5 -> ~b5

↪ b6 -> ~b6 b7 -> ~b7 b8 -> ~b8 ;

rup >= 1 ;

output NONE ;

conclusion UNSAT ;

end pseudo-Boolean proof ;

- Start the proof
- Derive $b_1 \geq 1$ with redundance rule

The Problem of Deleting Constraints

Important to allow deletions of constraints from database

- Improves practical performance
- Affects power of strengthening rules

- Improves practical performance
- Affects power of strengthening rules

- Unsatisfiable formulas can turn satisfiable
- Satisfiable formulas can turn unsatisfiable(!)

- Improves practical performance
- Affects power of strengthening rules

- Unsatisfiable formulas can turn satisfiable
- Satisfiable formulas can turn unsatisfiable(!)

Enumeration: deletion can reintroduce already enumerated solutions

- Improves practical performance
- Affects power of strengthening rules

- Unsatisfiable formulas can turn satisfiable
- Satisfiable formulas can turn unsatisfiable(!)

Enumeration: deletion can reintroduce already enumerated solutions

Solution: distinguish between deletion from core set $\mathcal{C}$ and derived set $\mathcal{D}$

Optimisation Problems

Any solution α found is logged with `sol i` “log solution and improve” command

- Provided solution α checked against current core set $\mathcal{C}$
- **Objective-improving constraint** $f \leq -1 + \sum_i w_i \cdot \alpha(\ell_i) + k$ added to core set (forces search for better solutions)

Note that

- α need not be solution for original formula
- but such solution can be reconstructed from the proof

Proof of optimality: Contradiction derived from objective-improving constraint

Proof format supports not just optimality, but also non-tight upper and lower bounds



Syntactic Sugar: Labels for Constraints

Constraint identifiers are consecutive numbers 1, 2, 3, ...

- Can be annoying to keep track of the exact numbers when writing proofs
- Or plain difficult if several (sub)solvers collaborate

Syntactic Sugar: Labels for Constraints

Constraint identifiers are consecutive numbers 1, 2, 3, ...

- Can be annoying to keep track of the exact numbers when writing proofs
- Or plain difficult if several (sub)solvers collaborate

Labels simplify things — just **aliases** `@<label>` for **constraint IDs** (in input file or proof file)

```
* input file (.opb)
```

```
@foo 1 x1 1 x2 >= 1 ;
```

```
@bar 1 x1 1 x3 >= 1 ;
```

```
@baz 1 x2 1 x3 >= 1 ;
```

```
% proof file (.pbp)
```

```
@sum pol @foo @bar + @baz + ;
```

```
pol @sum 2 d ;
```

Syntactic Sugar: Labels for Constraints

Constraint identifiers are consecutive numbers 1, 2, 3, ...

- Can be annoying to keep track of the exact numbers when writing proofs
- Or plain difficult if several (sub)solvers collaborate

Labels simplify things — just **aliases** `@<label>` for **constraint IDs** (in input file or proof file)

```
* input file (.opb)
```

```
@foo 1 x1 1 x2 >= 1 ;
```

```
@bar 1 x1 1 x3 >= 1 ;
```

```
@baz 1 x2 1 x3 >= 1 ;
```

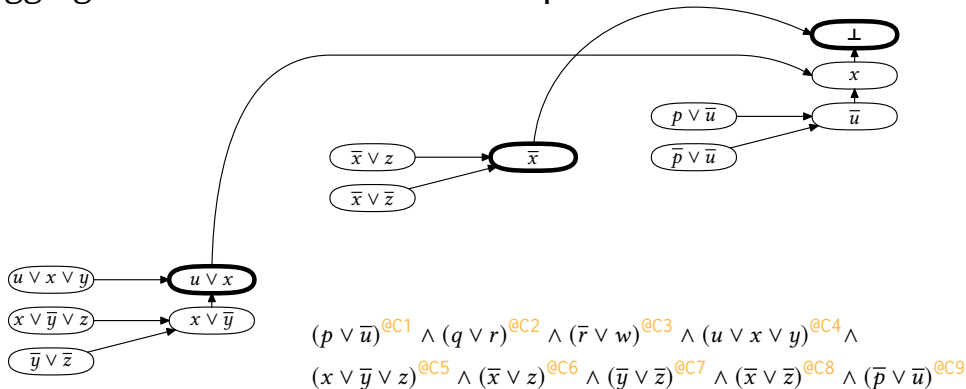
```
% proof file (.pbp)
```

```
@sum pol @foo @bar + @baz + ;
```

```
pol @sum 2 d ;
```

- Can be used anywhere a constraint ID is expected
- Prefixing of proof rule assigns label to derived constraint
- Defining a label again just **overwrites** it — can be convenient when referring to simplified form of “the same constraint”

Proof Logging with Labels for CDCL Example



```
@C10 pol @C7 @C5 + 2 d @C4 + 2 d ;
```

$$\rightsquigarrow u + x \geq 1$$

```
@C11 pol @C8 @C6 + 2 d ;
```

$$\rightsquigarrow \bar{x} \geq 1$$

```
@C12 pol @C9 @C1 + 2 d @C10 + 2 d @C11 + 2 d ;
```

$$\rightsquigarrow 0 \geq 1 \quad \text{⚡}$$

Linear Pseudo-Boolean Constraints and Normalised Form

Usually, **pseudo-Boolean constraints** are any **0–1 integer linear constraints** of the form

$$\sum_i a_i \ell_i \bowtie A$$

- $\bowtie \in \{\geq, \leq, =, >, <\}$
- $a_i, A \in \mathbb{Z}$
- **literals** ℓ_i : x_i or $\bar{x}_i$ (where $x_i + \bar{x}_i = 1$)
- variables x_i take values $0 = \text{false}$ or $1 = \text{true}$

Linear Pseudo-Boolean Constraints and Normalised Form

Usually, **pseudo-Boolean constraints** are any **0–1 integer linear constraints** of the form

$$\sum_i a_i \ell_i \bowtie A$$

- $\bowtie \in \{\geq, \leq, =, >, <\}$
- $a_i, A \in \mathbb{Z}$
- **literals** ℓ_i : x_i or $\bar{x}_i$ (where $x_i + \bar{x}_i = 1$)
- variables x_i take values $0 = \text{false}$ or $1 = \text{true}$

For us, convenient to use **normalised form** [Bar95] (without loss of generality)

$$\sum_i a_i \ell_i \geq A$$

- constraint always greater-than-or-equal
- $a_i, A \in \mathbb{N}$ non-negative
- $A = \text{deg}(\sum_i a_i \ell_i \geq A)$ referred to as **degree (of falsity)**

Conversion to Normalised Form: Example

Normalised form is used for convenience and without loss of generality

$$-x_1 + 2x_2 - 3x_3 + 4x_4 - 5x_5 < 0$$

1 Make inequality non-strict

$$-x_1 + 2x_2 - 3x_3 + 4x_4 - 5x_5 \leq -1$$

Conversion to Normalised Form: Example

Normalised form is used for convenience and without loss of generality

$$-x_1 + 2x_2 - 3x_3 + 4x_4 - 5x_5 < 0$$

- 1 Make inequality non-strict

$$-x_1 + 2x_2 - 3x_3 + 4x_4 - 5x_5 \leq -1$$

- 2 Multiply by -1 to get greater-than-or-equal

$$x_1 - 2x_2 + 3x_3 - 4x_4 + 5x_5 \geq 1$$

Conversion to Normalised Form: Example

Normalised form is used for convenience and without loss of generality

$$-x_1 + 2x_2 - 3x_3 + 4x_4 - 5x_5 < 0$$

- 1 Make inequality non-strict

$$-x_1 + 2x_2 - 3x_3 + 4x_4 - 5x_5 \leq -1$$

- 2 Multiply by -1 to get greater-than-or-equal

$$x_1 - 2x_2 + 3x_3 - 4x_4 + 5x_5 \geq 1$$

- 3 Replace $-\ell$ by $-(1 - \bar{\ell})$ [where we define $\bar{\bar{x}} \doteq x$]

$$x_1 - 2(1 - \bar{x}_2) + 3x_3 - 4(1 - \bar{x}_4) + 5x_5 \geq 1$$

$$x_1 + 2\bar{x}_2 + 3x_3 + 4\bar{x}_4 + 5x_5 \geq 7$$

Conversion to Normalised Form: Example

Normalised form is used for convenience and without loss of generality

$$-x_1 + 2x_2 - 3x_3 + 4x_4 - 5x_5 < 0$$

- 1 Make inequality non-strict

$$-x_1 + 2x_2 - 3x_3 + 4x_4 - 5x_5 \leq -1$$

- 2 Multiply by -1 to get greater-than-or-equal

$$x_1 - 2x_2 + 3x_3 - 4x_4 + 5x_5 \geq 1$$

- 3 Replace $-\ell$ by $-(1 - \bar{\ell})$ [where we define $\bar{\bar{x}} \doteq x$]

$$x_1 - 2(1 - \bar{x}_2) + 3x_3 - 4(1 - \bar{x}_4) + 5x_5 \geq 1$$

$$x_1 + 2\bar{x}_2 + 3x_3 + 4\bar{x}_4 + 5x_5 \geq 7$$

- 4 Replace “=” by two inequalities “ $\geq$ ” and “ $\leq$ ”

Conversion to Normalised Form: Formal Details

Given (normalised) linear form $\sum_i a_i \ell_i$ with coefficient sum $\sum_i a_i = W$

Syntactic sugar

$$\sum_i a_i \ell_i > A$$

$$\sum_i a_i \ell_i \leq A$$

$$\sum_i a_i \ell_i < A$$

$$\sum_i a_i \ell_i = A$$

Meaning

$$\sum_i a_i \ell_i \geq A + 1$$

$$\sum_i a_i \bar{\ell}_i \geq W - A$$

$$\sum_i a_i \bar{\ell}_i \geq W - A + 1$$

$$\sum_i a_i \ell_i \geq A \quad \text{and}$$

$$\sum_i a_i \bar{\ell}_i \geq W - A$$

Conversion to Normalised Form: Formal Details

Given (normalised) linear form $\sum_i a_i \ell_i$ with coefficient sum $\sum_i a_i = W$

Syntactic sugar

$$\sum_i a_i \ell_i > A$$

$$\sum_i a_i \ell_i \leq A$$

$$\sum_i a_i \ell_i < A$$

$$\sum_i a_i \ell_i = A$$

Meaning

$$\sum_i a_i \ell_i \geq A + 1$$

$$\sum_i a_i \bar{\ell}_i \geq W - A$$

$$\sum_i a_i \bar{\ell}_i \geq W - A + 1$$

$$\sum_i a_i \ell_i \geq A \quad \text{and}$$

$$\sum_i a_i \bar{\ell}_i \geq W - A$$

In what follows:

- Use syntactic sugar when convenient
- Assume (implicit) normalisation whenever it matters
- Write $\doteq$ for syntactic equality

Some Convenient Notation for Constraints

Given constraint $C \doteq \sum_i a_i \ell_i \geq A$ with $\sum_i a_i = W$

Negation

$$\neg C \doteq \sum_i a_i \bar{\ell}_i \geq W - A + 1$$

Some Convenient Notation for Constraints

Given constraint $C \doteq \sum_i a_i \ell_i \geq A$ with $\sum_i a_i = W$

Negation

$$\neg C \doteq \sum_i a_i \bar{\ell}_i \geq W - A + 1$$

Reification

$$r \Rightarrow C \doteq A \cdot \bar{r} + \sum_i a_i \ell_i \geq A$$

$$r \Leftarrow C \doteq (W - A + 1) \cdot r + \sum_i a_i \bar{\ell}_i \geq W - A + 1$$

$$r \Leftrightarrow C \doteq r \Rightarrow C \text{ and } r \Leftarrow C$$

Some Convenient Notation for Constraints

Given constraint $C \doteq \sum_i a_i \ell_i \geq A$ with $\sum_i a_i = W$

Negation

$$\neg C \doteq \sum_i a_i \bar{\ell}_i \geq W - A + 1$$

Reification

$$r \Rightarrow C \doteq A \cdot \bar{r} + \sum_i a_i \ell_i \geq A$$

$$r \Leftarrow C \doteq (W - A + 1) \cdot r + \sum_i a_i \bar{\ell}_i \geq W - A + 1$$

$$r \Leftrightarrow C \doteq r \Rightarrow C \text{ and } r \Leftarrow C$$

Some calculations

$$r \Leftarrow C \doteq \bar{r} \Rightarrow \neg C$$

$$C + \neg C \doteq 0 \geq 1$$

$$\deg(C) \cdot (r \geq 1) + (r \Rightarrow C) \doteq C$$

$$C + (r \Leftarrow C) \doteq \deg(\neg C) \cdot r \geq 1$$

More Syntactic Sugar: Reification in VERIPB Syntax

For big-M reification constraints, can be annoying to compute the right constant

So VERIPB allows **writing reified constraints with arrows**

$r \implies \langle \text{constraint} \rangle ;$

$r \leq \langle \text{constraint} \rangle ;$

$r \iff \langle \text{constraint} \rangle ;$

More Syntactic Sugar: Reification in VERIPB Syntax

For big-M reification constraints, can be annoying to compute the right constant

So VERIPB allows **writing reified constraints with arrows**

$r \implies \langle \text{constraint} \rangle ;$

$r \leq \langle \text{constraint} \rangle ;$

$r \iff \langle \text{constraint} \rangle ;$

- $\iff$ stands for two constraints and lives in the input file only
- $\leq$ and $\iff$ take **exactly one** literal on the left
- $\implies$ allows **one or more** literals, read as their **conjunction**

More Syntactic Sugar: Reification in VERIPB Syntax

For big-M reification constraints, can be annoying to compute the right constant

So VERIPB allows **writing reified constraints with arrows**

$r \implies \langle \text{constraint} \rangle ;$

$r \leq \langle \text{constraint} \rangle ;$

$r \iff \langle \text{constraint} \rangle ;$

- $\iff$ stands for two constraints and lives in the input file only
- $\leq$ and $\iff$ take **exactly one** literal on the left
- $\implies$ allows **one or more** literals, read as their **conjunction**

For $C \doteq \sum_i a_i \ell_i \geq A$ and literals $r_1, r_2, \dots, r_n$, we have

$$r_1 \wedge r_2 \wedge \dots \wedge r_n \implies C \doteq A\bar{r}_1 + A\bar{r}_2 + \dots + A\bar{r}_n + \sum_i a_i \ell_i \geq A$$

Reification in VERIPB Syntax: Concrete Examples

For $C \doteq x_1 + 2x_2 + 3x_3 + 4x_4 \geq 5$ we have

- $\text{deg}(C) = 5$
- $W = \sum_i a_i = 10$
- $\neg C \doteq \bar{x}_1 + 2\bar{x}_2 + 3\bar{x}_3 + 4\bar{x}_4 \geq 10 - 5 + 1$

Reification in VERIPB Syntax: Concrete Examples

For $C \doteq x_1 + 2x_2 + 3x_3 + 4x_4 \geq 5$ we have

- $\text{deg}(C) = 5$
- $W = \sum_i a_i = 10$
- $\neg C \doteq \bar{x}_1 + 2\bar{x}_2 + 3\bar{x}_3 + 4\bar{x}_4 \geq 6$

Reification in VERIPB Syntax: Concrete Examples

For $C \doteq x_1 + 2x_2 + 3x_3 + 4x_4 \geq 5$ we have

- $\text{deg}(C) = 5$
- $W = \sum_i a_i = 10$
- $\neg C \doteq \bar{x}_1 + 2\bar{x}_2 + 3\bar{x}_3 + 4\bar{x}_4 \geq 6$

Syntactic sugar

$r \implies 1 \ x_1 \ 2 \ x_2 \ 3 \ x_3 \ 4 \ x_4 \geq 5 ;$

$r \leq 1 \ x_1 \ 2 \ x_2 \ 3 \ x_3 \ 4 \ x_4 \geq 5 ;$

$r \iff 1 \ x_1 \ 2 \ x_2 \ 3 \ x_3 \ 4 \ x_4 \geq 5 ;$

$r_1 \sim r_2 \ r_3 \implies 1 \ x_1 \ 2 \ x_2 \ 3 \ x_3 \ 4 \ x_4 \geq 5 ;$

Meaning

$5\bar{r} + x_1 + 2x_2 + 3x_3 + 4x_4 \geq 5$

$6r + \bar{x}_1 + 2\bar{x}_2 + 3\bar{x}_3 + 4\bar{x}_4 \geq 6$

1st and 2nd constraint, in that order

$5\bar{r}_1 + 5r_2 + 5\bar{r}_3 + x_1 + 2x_2 + 3x_3 + 4x_4 \geq 5$

Reification in VERIPB Syntax: Concrete Examples

For $C \doteq x_1 + 2x_2 + 3x_3 + 4x_4 \geq 5$ we have

- $\text{deg}(C) = 5$
- $W = \sum_i a_i = 10$
- $\neg C \doteq \bar{x}_1 + 2\bar{x}_2 + 3\bar{x}_3 + 4\bar{x}_4 \geq 6$

Syntactic sugar

$r \implies 1 \ x_1 \ 2 \ x_2 \ 3 \ x_3 \ 4 \ x_4 \geq 5 ;$

$r \leq 1 \ x_1 \ 2 \ x_2 \ 3 \ x_3 \ 4 \ x_4 \geq 5 ;$

$r \iff 1 \ x_1 \ 2 \ x_2 \ 3 \ x_3 \ 4 \ x_4 \geq 5 ;$ 1st and 2nd constraint, in that order

$r_1 \sim r_2 \ r_3 \implies 1 \ x_1 \ 2 \ x_2 \ 3 \ x_3 \ 4 \ x_4 \geq 5 ;$ $5\bar{r}_1 + 5r_2 + 5\bar{r}_3 + x_1 + 2x_2 + 3x_3 + 4x_4 \geq 5$

Meaning

$5\bar{r} + x_1 + 2x_2 + 3x_3 + 4x_4 \geq 5$

$6r + \bar{x}_1 + 2\bar{x}_2 + 3\bar{x}_3 + 4\bar{x}_4 \geq 6$

- Right reification: **negate reification literal(s)** and add with **coefficient $\text{deg}(C) = 5$**
- Left reification: **negate constraint** and add reification literal with **coefficient $\text{deg}(\neg C) = 6$**

Calculations with Reified Constraints: Concrete Examples

Still with $C \doteq x_1 + 2x_2 + 3x_3 + 4x_4 \geq 5$, so that $\neg C \doteq \bar{x}_1 + 2\bar{x}_2 + 3\bar{x}_3 + 4\bar{x}_4 \geq 6$
(and recall that $\ell + \bar{\ell} = 1$, so that opposing literals cancel and leave 1)

$$\begin{aligned} C + \neg C &\doteq (x_1 + \bar{x}_1) + 2(x_2 + \bar{x}_2) + 3(x_3 + \bar{x}_3) + 4(x_4 + \bar{x}_4) \geq 11 \\ &\doteq 0 \geq 11 - 10 \\ &\doteq 0 \geq 1 \end{aligned}$$

$$\begin{aligned} \text{deg}(C) \cdot (r \geq 1) + (r \Rightarrow C) &\doteq 5(r + \bar{r}) + x_1 + 2x_2 + 3x_3 + 4x_4 \geq 10 \\ &\doteq x_1 + 2x_2 + 3x_3 + 4x_4 \geq 10 - 5 \\ &\doteq C \end{aligned}$$

$$\begin{aligned} C + (r \Leftarrow C) &\doteq 6r + (x_1 + \bar{x}_1) + 2(x_2 + \bar{x}_2) + 3(x_3 + \bar{x}_3) + 4(x_4 + \bar{x}_4) \geq 11 \\ &\doteq 6r \geq 11 - 10 \\ &\doteq \text{deg}(\neg C) \cdot r \geq 1 \end{aligned}$$

Cutting Planes Derivations Under Assumptions

Suppose we have the constraints

$$r_1 + x_1 + x_2 \geq 1$$

$$r_1 + x_1 + x_3 \geq 1$$

$$x_2 + x_3 \geq 1$$

$$\bar{r}_2 + \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2$$

Cutting Planes Derivations Under Assumptions

Suppose we have the constraints

$$\cancel{r_1} + x_1 + x_2 \geq 1 \quad \rightsquigarrow \quad x_1 + x_2 \geq 1$$

$$\cancel{r_1} + x_1 + x_3 \geq 1 \quad \rightsquigarrow \quad x_1 + x_3 \geq 1$$

$$x_2 + x_3 \geq 1 \quad \rightsquigarrow \quad x_2 + x_3 \geq 1$$

$$\cancel{r_2} + \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2 \quad \rightsquigarrow \quad \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2$$

Assume $r_1 = 0$ and $r_2 = 1$ and simplify

Cutting Planes Derivations Under Assumptions

Suppose we have the constraints

$$\cancel{r_1} + x_1 + x_2 \geq 1 \rightsquigarrow x_1 + x_2 \geq 1$$

$$\cancel{r_1} + x_1 + x_3 \geq 1 \rightsquigarrow x_1 + x_3 \geq 1$$

$$x_2 + x_3 \geq 1 \rightsquigarrow x_2 + x_3 \geq 1$$

$$\cancel{r_2} + \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2 \rightsquigarrow \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2$$

Assume $r_1 = 0$ and $r_2 = 1$ and simplify — then cutting planes can derive contradiction

$$\begin{array}{rcl}
 \text{Add} & \frac{x_1 + x_2 \geq 1 \quad x_1 + x_3 \geq 1}{2x_1 + x_2 + x_3 \geq 2} & x_2 + x_3 \geq 1 \\
 \text{Add} & \frac{2x_1 + x_2 + x_3 \geq 2 \quad x_2 + x_3 \geq 1}{2x_1 + 2x_2 + 2x_3 \geq 3} & \\
 \text{Divide by 2} & \frac{2x_1 + 2x_2 + 2x_3 \geq 3}{x_1 + x_2 + x_3 \geq 2} & \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2 \\
 \text{Add} & \frac{x_1 + x_2 + x_3 \geq 2 \quad \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2}{0 \geq 1} &
 \end{array}$$

Cutting Planes Derivations Under Assumptions

Suppose we have the constraints

$$\cancel{r_1} + x_1 + x_2 \geq 1 \quad \rightsquigarrow \quad x_1 + x_2 \geq 1$$

$$\cancel{r_1} + x_1 + x_3 \geq 1 \quad \rightsquigarrow \quad x_1 + x_3 \geq 1$$

$$x_2 + x_3 \geq 1 \quad \rightsquigarrow \quad x_2 + x_3 \geq 1$$

$$\cancel{\bar{r}_2} + \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2 \quad \rightsquigarrow \quad \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2$$

Assume $r_1 = 0$ and $r_2 = 1$ and simplify — then cutting planes can derive contradiction

$$\begin{array}{rcl}
 \text{Add} & \frac{x_1 + x_2 \geq 1 \quad x_1 + x_3 \geq 1}{2x_1 + x_2 + x_3 \geq 2} & x_2 + x_3 \geq 1 \\
 \text{Add} & \frac{2x_1 + x_2 + x_3 \geq 2 \quad x_2 + x_3 \geq 1}{2x_1 + 2x_2 + 2x_3 \geq 3} & \\
 \text{Divide by 2} & \frac{2x_1 + 2x_2 + 2x_3 \geq 3}{x_1 + x_2 + x_3 \geq 2} & \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2 \\
 \text{Add} & \frac{x_1 + x_2 + x_3 \geq 2 \quad \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2}{0 \geq 1} &
 \end{array}$$

This means that the original constraints imply $r_1 + \bar{r}_2 \geq 1$

The Same Derivation Without Assumptions

Now **forget about the assumptions** and **run exactly the same derivation** on the original constraints:

$$\begin{array}{l}
 \text{Add} \quad \frac{r_1 + x_1 + x_2 \geq 1 \quad r_1 + x_1 + x_3 \geq 1}{2r_1 + 2x_1 + x_2 + x_3 \geq 2 \quad x_2 + x_3 \geq 1} \\
 \text{Add} \quad \frac{2r_1 + 2x_1 + x_2 + x_3 \geq 2 \quad x_2 + x_3 \geq 1}{2r_1 + 2x_1 + 2x_2 + 2x_3 \geq 3} \\
 \text{Divide by 2} \quad \frac{2r_1 + 2x_1 + 2x_2 + 2x_3 \geq 3}{r_1 + x_1 + x_2 + x_3 \geq 2} \\
 \text{Add} \quad \frac{r_1 + x_1 + x_2 + x_3 \geq 2 \quad \bar{r}_2 + \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2}{r_1 + \bar{r}_2 \geq 1}
 \end{array}$$

The Same Derivation Without Assumptions

Now **forget about the assumptions** and **run exactly the same derivation** on the original constraints:

$$\begin{array}{l}
 \text{Add} \quad \frac{r_1 + x_1 + x_2 \geq 1 \quad r_1 + x_1 + x_3 \geq 1}{2r_1 + 2x_1 + x_2 + x_3 \geq 2} \quad x_2 + x_3 \geq 1 \\
 \text{Add} \quad \frac{2r_1 + 2x_1 + x_2 + x_3 \geq 2 \quad x_2 + x_3 \geq 1}{2r_1 + 2x_1 + 2x_2 + 2x_3 \geq 3} \\
 \text{Divide by 2} \quad \frac{2r_1 + 2x_1 + 2x_2 + 2x_3 \geq 3}{r_1 + x_1 + x_2 + x_3 \geq 2} \\
 \text{Add} \quad \frac{r_1 + x_1 + x_2 + x_3 \geq 2 \quad \bar{r}_2 + \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2}{r_1 + \bar{r}_2 \geq 1}
 \end{array}$$

This yields

$$r_1 + \bar{r}_2 \geq 1 \quad [\text{VERIPB syntax: } 1 \ r1 \ 1 \ \sim r2 \ >= \ 1 \ ;]$$

or equivalently

$$\bar{r}_1 \wedge r_2 \Rightarrow (0 \geq 1) \quad [\text{VERIPB syntax: } \sim r1 \ r2 \ ==> \ >= \ 1 \ ;]$$

The Same Derivation Without Assumptions

Now **forget about the assumptions** and **run exactly the same derivation** on the original constraints:

$$\begin{array}{l}
 \text{Add} \quad \frac{r_1 + x_1 + x_2 \geq 1 \quad r_1 + x_1 + x_3 \geq 1}{2r_1 + 2x_1 + x_2 + x_3 \geq 2} \quad x_2 + x_3 \geq 1 \\
 \text{Add} \quad \frac{2r_1 + 2x_1 + x_2 + x_3 \geq 2 \quad x_2 + x_3 \geq 1}{2r_1 + 2x_1 + 2x_2 + 2x_3 \geq 3} \\
 \text{Divide by 2} \quad \frac{2r_1 + 2x_1 + 2x_2 + 2x_3 \geq 3}{r_1 + x_1 + x_2 + x_3 \geq 2} \quad \bar{r}_2 + \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2 \\
 \text{Add} \quad \frac{r_1 + x_1 + x_2 + x_3 \geq 2 \quad \bar{r}_2 + \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2}{r_1 + \bar{r}_2 \geq 1}
 \end{array}$$

This yields

$$r_1 + \bar{r}_2 \geq 1 \quad [\text{VERIPB syntax: } 1 \ r1 \ 1 \ \sim r2 \ >= \ 1 \ ;]$$

or equivalently

$$\bar{r}_1 \wedge r_2 \Rightarrow (0 \geq 1) \quad [\text{VERIPB syntax: } \sim r1 \ r2 \ ==> \ >= \ 1 \ ;]$$

That is, our assumptions imply contradiction

This works in general!

Derivations Under Assumptions: The General Claim

This works in general!

■ Suppose that we have

- 1 constraints $C_1, C_2, \dots, C_m$
- 2 assumptions $\rho = \{\ell_1 = 1, \ell_2 = 1, \dots, \ell_n = 1\}$
- 3 cutting planes derivation of D from the restricted constraints $C_1 \upharpoonright_\rho, C_2 \upharpoonright_\rho, \dots, C_m \upharpoonright_\rho$
(i.e., derivation without strengthening)

Derivations Under Assumptions: The General Claim

This works in general!

- Suppose that we have
 - 1 constraints $C_1, C_2, \dots, C_m$
 - 2 assumptions $\rho = \{\ell_1 = 1, \ell_2 = 1, \dots, \ell_n = 1\}$
 - 3 cutting planes derivation of D from the restricted constraints $C_1 \upharpoonright_\rho, C_2 \upharpoonright_\rho, \dots, C_m \upharpoonright_\rho$ (i.e., derivation without strengthening)
- Then the same derivation applied to $C_1, C_2, \dots, C_m$ yields a constraint at least as strong as

$$\ell_1 \wedge \ell_2 \wedge \dots \wedge \ell_n \Rightarrow D \doteq \deg(D) \cdot \sum_{i=1}^n \bar{\ell}_i + D$$

Derivations Under Assumptions: The General Claim

This works in general!

- Suppose that we have

- 1 constraints $C_1, C_2, \dots, C_m$

- 2 assumptions $\rho = \{\ell_1 = 1, \ell_2 = 1, \dots, \ell_n = 1\}$

- 3 cutting planes derivation of D from the restricted constraints $C_1 \upharpoonright_\rho, C_2 \upharpoonright_\rho, \dots, C_m \upharpoonright_\rho$ (i.e., derivation without strengthening)

- Then the same derivation applied to $C_1, C_2, \dots, C_m$ yields a constraint at least as strong as

$$\ell_1 \wedge \ell_2 \wedge \dots \wedge \ell_n \Rightarrow D \doteq \deg(D) \cdot \sum_{i=1}^n \bar{\ell}_i + D$$

Gives a convenient way to construct derivations:

- Apply assumptions in ρ to constraints C_j and simplify to get $C_j \upharpoonright_\rho$

Derivations Under Assumptions: The General Claim

This works in general!

- Suppose that we have
 - 1 constraints $C_1, C_2, \dots, C_m$
 - 2 assumptions $\rho = \{\ell_1 = 1, \ell_2 = 1, \dots, \ell_n = 1\}$
 - 3 cutting planes derivation of D from the restricted constraints $C_1 \upharpoonright_\rho, C_2 \upharpoonright_\rho, \dots, C_m \upharpoonright_\rho$ (i.e., derivation without strengthening)
- Then the same derivation applied to $C_1, C_2, \dots, C_m$ yields a constraint at least as strong as

$$\ell_1 \wedge \ell_2 \wedge \dots \wedge \ell_n \Rightarrow D \doteq \deg(D) \cdot \sum_{i=1}^n \bar{\ell}_i + D$$

Gives a convenient way to construct derivations:

- Apply assumptions in ρ to constraints C_j and simplify to get $C_j \upharpoonright_\rho$
- Derive desired conclusion D

Derivations Under Assumptions: The General Claim

This works in general!

- Suppose that we have
 - 1 constraints $C_1, C_2, \dots, C_m$
 - 2 assumptions $\rho = \{\ell_1 = 1, \ell_2 = 1, \dots, \ell_n = 1\}$
 - 3 cutting planes derivation of D from the restricted constraints $C_1 \upharpoonright_\rho, C_2 \upharpoonright_\rho, \dots, C_m \upharpoonright_\rho$ (i.e., derivation without strengthening)
- Then the same derivation applied to $C_1, C_2, \dots, C_m$ yields a constraint at least as strong as

$$\ell_1 \wedge \ell_2 \wedge \dots \wedge \ell_n \Rightarrow D \doteq \deg(D) \cdot \sum_{i=1}^n \bar{\ell}_i + D$$

Gives a convenient way to construct derivations:

- Apply assumptions in ρ to constraints C_j and simplify to get $C_j \upharpoonright_\rho$
- Derive desired conclusion D
- Remove assumptions to get reified version $\rho \Rightarrow D$ of constraint

Derivations Under Assumptions: The General Claim

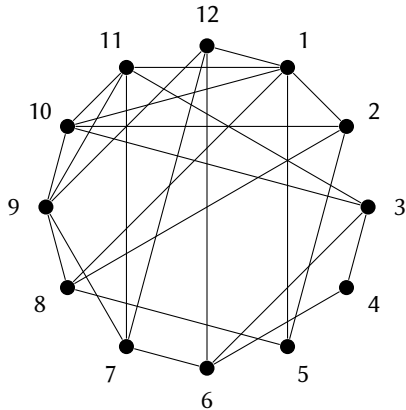
This works in general!

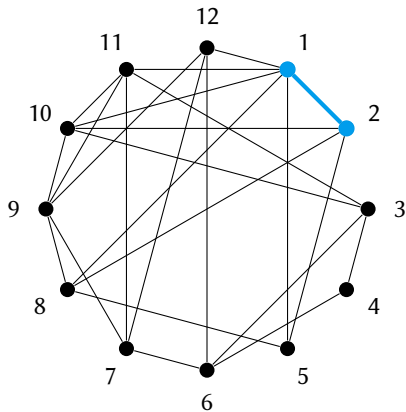
- Suppose that we have
 - 1 constraints $C_1, C_2, \dots, C_m$
 - 2 assumptions $\rho = \{\ell_1 = 1, \ell_2 = 1, \dots, \ell_n = 1\}$
 - 3 cutting planes derivation of D from the restricted constraints $C_1 \upharpoonright_\rho, C_2 \upharpoonright_\rho, \dots, C_m \upharpoonright_\rho$ (i.e., derivation without strengthening)
- Then the same derivation applied to $C_1, C_2, \dots, C_m$ yields a constraint at least as strong as

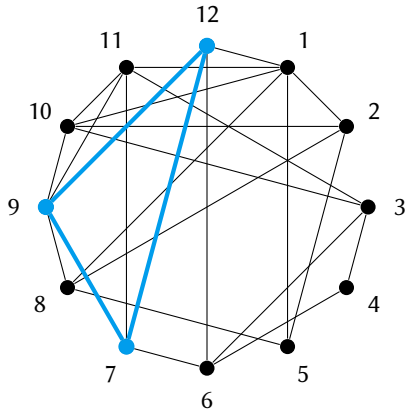
$$\ell_1 \wedge \ell_2 \wedge \dots \wedge \ell_n \Rightarrow D \doteq \deg(D) \cdot \sum_{i=1}^n \bar{\ell}_i + D$$

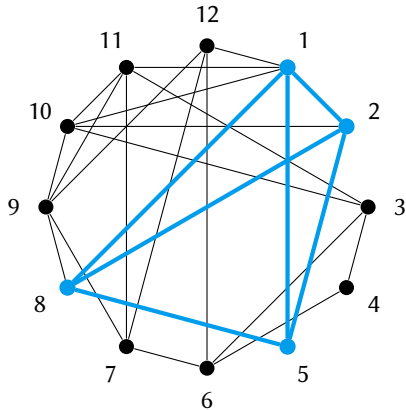
Gives a convenient way to construct derivations:

- Apply assumptions in ρ to constraints C_j and simplify to get $C_j \upharpoonright_\rho$
- Derive desired conclusion D
- Remove assumptions to get reified version $\rho \Rightarrow D$ of constraint
- Important special case: Backtracking clause from decisions/guesses leading to contradiction

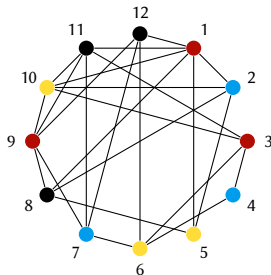








A Brief and Incomplete Guide to Clique Solving (3/4)



Given a k -colouring of a subgraph, that subgraph cannot have a clique of more than k vertices

We can use $|C| + \#colours(P)$ as a bound, for any colouring

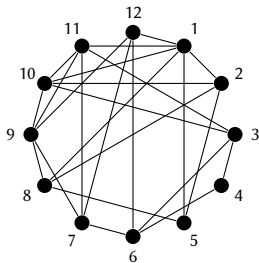
A Brief and Incomplete Guide to Clique Solving (4/4)

- This brings us to 1997
- Many improvements since then
 - better bound functions
 - clever vertex selection heuristics
 - efficient data structures
 - local search
 - ...
- But key ideas for proof logging can be explained without worrying about such things

Making a Certifying Clique Solver

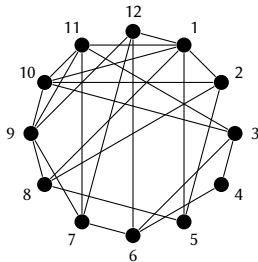
- 1 Output a pseudo-Boolean encoding of the problem
 - Clique problems have several standard file formats
 - Give the constraints labels, so the proof can refer to them by name
- 2 Make solver generate proof log for search tree
 - Output a small header
 - Output something on every backtrack
 - Output something every time a solution is found
 - Output a small footer
- 3 Figure out how to certify the bound function

A Pseudo-Boolean Encoding for Clique (in OPB Format)



```
* #variable= 12 #constraint= 41
min: 1 ~x1 1 ~x2 1 ~x3 1 ~x4 . . . and so on . . . 1 ~x11 1 ~x12 ;
@noedge1_3 -1 x3 -1 x1 >= -1 ;
@noedge2_3 -1 x3 -1 x2 >= -1 ;
@noedge1_4 -1 x4 -1 x1 >= -1 ;
* . . . and a further 38 similar lines for the remaining non-edges
```

A Pseudo-Boolean Encoding for Clique (in OPB Format)



```
* #variable= 12 #constraint= 41
```

```
min: 1 ~x1 1 ~x2 1 ~x3 1 ~x4 . . . and so on . . . 1 ~x11 1 ~x12 ;
```

```
@noedge1_3 -1 x3 -1 x1 >= -1 ;
```

```
@noedge2_3 -1 x3 -1 x2 >= -1 ;
```

```
@noedge1_4 -1 x4 -1 x1 >= -1 ;
```

```
* . . . and a further 38 similar lines for the remaining non-edges
```

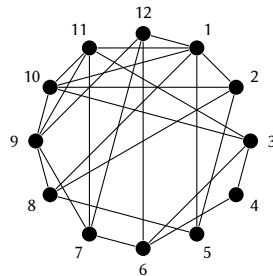
$x_i = 1$ means “vertex i is in the clique”

VERIPB proof minimises number of vertices **not** selected, so 4-clique yields objective value $12 - 4 = 8$

First Attempt at a Proof

pseudo-Boolean proof version 3.0

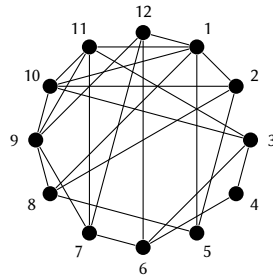
```
@obj soli x7 x9 x12 ;  
rup 1 ~x12 1 ~x7 >= 1 ;  
rup 1 ~x12 >= 1 ;  
rup 1 ~x11 1 ~x10 >= 1 ;  
rup 1 ~x11 >= 1 ;  
@obj soli x1 x2 x5 x8 ;  
rup 1 ~x8 1 ~x5 >= 1 ;  
rup 1 ~x8 >= 1 ;  
rup >= 1 ;  
output NONE ;  
conclusion BOUNDS 8 8 ;  
end pseudo-Boolean proof ;
```



First Attempt at a Proof

pseudo-Boolean proof version 3.0

```
@obj soli x7 x9 x12 ;  
rup 1 ~x12 1 ~x7 >= 1 ;  
rup 1 ~x12 >= 1 ;  
rup 1 ~x11 1 ~x10 >= 1 ;  
rup 1 ~x11 >= 1 ;  
@obj soli x1 x2 x5 x8 ;  
rup 1 ~x8 1 ~x5 >= 1 ;  
rup 1 ~x8 >= 1 ;  
rup >= 1 ;  
output NONE ;  
conclusion BOUNDS 8 8 ;  
end pseudo-Boolean proof ;
```



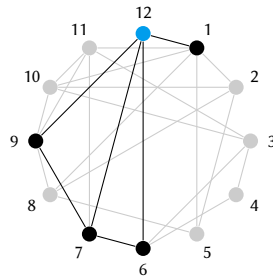
Start with a header

The problem constraints are loaded for us

First Attempt at a Proof

pseudo-Boolean proof version 3.0

```
@obj soli x7 x9 x12 ;  
rup 1 ~x12 1 ~x7 >= 1 ;  
rup 1 ~x12 >= 1 ;  
rup 1 ~x11 1 ~x10 >= 1 ;  
rup 1 ~x11 >= 1 ;  
@obj soli x1 x2 x5 x8 ;  
rup 1 ~x8 1 ~x5 >= 1 ;  
rup 1 ~x8 >= 1 ;  
rup >= 1 ;  
output NONE ;  
conclusion BOUNDS 8 8 ;  
end pseudo-Boolean proof ;
```

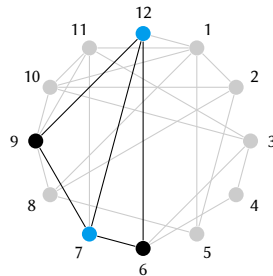


Branch accepting 12
Throw away non-adjacent vertices

First Attempt at a Proof

pseudo-Boolean proof version 3.0

```
@obj soli x7 x9 x12 ;  
rup 1 ~x12 1 ~x7 >= 1 ;  
rup 1 ~x12 >= 1 ;  
rup 1 ~x11 1 ~x10 >= 1 ;  
rup 1 ~x11 >= 1 ;  
@obj soli x1 x2 x5 x8 ;  
rup 1 ~x8 1 ~x5 >= 1 ;  
rup 1 ~x8 >= 1 ;  
rup >= 1 ;  
output NONE ;  
conclusion BOUNDS 8 8 ;  
end pseudo-Boolean proof ;
```

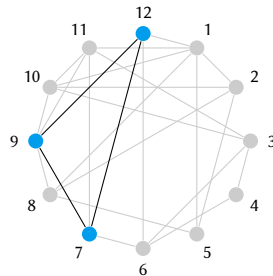


Branch also accepting 7
Throw away non-adjacent vertices

First Attempt at a Proof

pseudo-Boolean proof version 3.0

```
@obj soli x7 x9 x12 ;  
rup 1 ~x12 1 ~x7 >= 1 ;  
rup 1 ~x12 >= 1 ;  
rup 1 ~x11 1 ~x10 >= 1 ;  
rup 1 ~x11 >= 1 ;  
@obj soli x1 x2 x5 x8 ;  
rup 1 ~x8 1 ~x5 >= 1 ;  
rup 1 ~x8 >= 1 ;  
rup >= 1 ;  
output NONE ;  
conclusion BOUNDS 8 8 ;  
end pseudo-Boolean proof ;
```

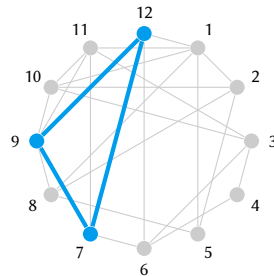


Branch also accepting 9
Throw away non-adjacent vertices

First Attempt at a Proof

pseudo-Boolean proof version 3.0

```
@obj soli x7 x9 x12 ;  
rup 1 ~x12 1 ~x7 >= 1 ;  
rup 1 ~x12 >= 1 ;  
rup 1 ~x11 1 ~x10 >= 1 ;  
rup 1 ~x11 >= 1 ;  
@obj soli x1 x2 x5 x8 ;  
rup 1 ~x8 1 ~x5 >= 1 ;  
rup 1 ~x8 >= 1 ;  
rup >= 1 ;  
output NONE ;  
conclusion BOUNDS 8 8 ;  
end pseudo-Boolean proof ;
```



We branched on 12, 7, 9
Found a new incumbent
Now looking for a ≥ 4 vertex clique
“@obj” names the constraint this creates

First Attempt at a Proof

pseudo-Boolean proof version 3.0

```
@obj soli x7 x9 x12 ;
```

```
rup 1 ~x12 1 ~x7 >= 1 ;
```

```
rup 1 ~x12 >= 1 ;
```

```
rup 1 ~x11 1 ~x10 >= 1 ;
```

```
rup 1 ~x11 >= 1 ;
```

```
@obj soli x1 x2 x5 x8 ;
```

```
rup 1 ~x8 1 ~x5 >= 1 ;
```

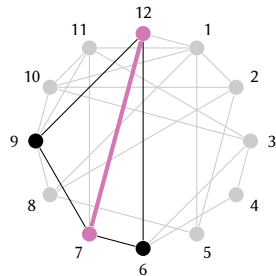
```
rup 1 ~x8 >= 1 ;
```

```
rup >= 1 ;
```

```
output NONE ;
```

```
conclusion BOUNDS 8 8 ;
```

```
end pseudo-Boolean proof ;
```

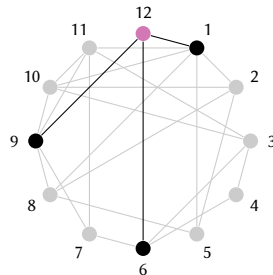


Backtrack from 12, 7
9 explored already, only 6 feasible
No ≥ 4 vertex clique possible
Effectively this deletes the 7–12 edge

First Attempt at a Proof

pseudo-Boolean proof version 3.0

```
@obj soli x7 x9 x12 ;  
rup 1 ~x12 1 ~x7 >= 1 ;  
rup 1 ~x12 >= 1 ;  
rup 1 ~x11 1 ~x10 >= 1 ;  
rup 1 ~x11 >= 1 ;  
@obj soli x1 x2 x5 x8 ;  
rup 1 ~x8 1 ~x5 >= 1 ;  
rup 1 ~x8 >= 1 ;  
rup >= 1 ;  
output NONE ;  
conclusion BOUNDS 8 8 ;  
end pseudo-Boolean proof ;
```

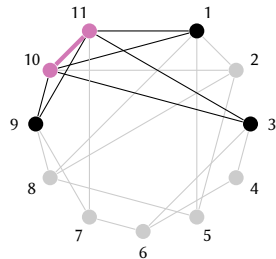


Backtrack from 12
Only 1, 6 and 9 feasible (1-colourable)
No ≥ 4 vertex clique possible
Effectively this deletes vertex 12

First Attempt at a Proof

pseudo-Boolean proof version 3.0

```
@obj soli x7 x9 x12 ;  
rup 1 ~x12 1 ~x7 >= 1 ;  
rup 1 ~x12 >= 1 ;  
rup 1 ~x11 1 ~x10 >= 1 ;  
rup 1 ~x11 >= 1 ;  
@obj soli x1 x2 x5 x8 ;  
rup 1 ~x8 1 ~x5 >= 1 ;  
rup 1 ~x8 >= 1 ;  
rup >= 1 ;  
output NONE ;  
conclusion BOUNDS 8 8 ;  
end pseudo-Boolean proof ;
```

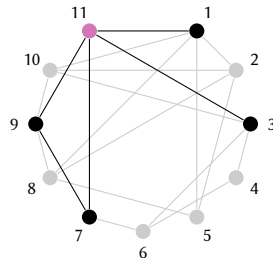


Branch on 11 then 10
Only 1, 3 and 9 feasible (1-colourable)
No ≥ 4 vertex clique possible
Backtrack, deleting the edge

First Attempt at a Proof

pseudo-Boolean proof version 3.0

```
@obj soli x7 x9 x12 ;  
rup 1 ~x12 1 ~x7 >= 1 ;  
rup 1 ~x12 >= 1 ;  
rup 1 ~x11 1 ~x10 >= 1 ;  
rup 1 ~x11 >= 1 ;  
@obj soli x1 x2 x5 x8 ;  
rup 1 ~x8 1 ~x5 >= 1 ;  
rup 1 ~x8 >= 1 ;  
rup >= 1 ;  
output NONE ;  
conclusion BOUNDS 8 8 ;  
end pseudo-Boolean proof ;
```

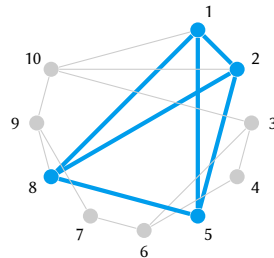


Backtrack from 11
2-colourable, so no ≥ 4 clique
Delete the vertex

First Attempt at a Proof

pseudo-Boolean proof version 3.0

```
@obj soli x7 x9 x12 ;  
rup 1 ~x12 1 ~x7 >= 1 ;  
rup 1 ~x12 >= 1 ;  
rup 1 ~x11 1 ~x10 >= 1 ;  
rup 1 ~x11 >= 1 ;  
@obj soli x1 x2 x5 x8 ;  
rup 1 ~x8 1 ~x5 >= 1 ;  
rup 1 ~x8 >= 1 ;  
rup >= 1 ;  
output NONE ;  
conclusion BOUNDS 8 8 ;  
end pseudo-Boolean proof ;
```

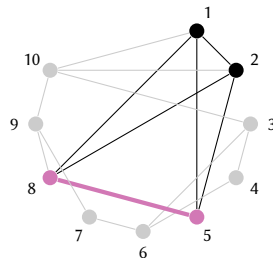


Branch on 8, 5, 1, 2
Found a new incumbent
Now looking for a ≥ 5 vertex clique

First Attempt at a Proof

pseudo-Boolean proof version 3.0

```
@obj soli x7 x9 x12 ;  
rup 1 ~x12 1 ~x7 >= 1 ;  
rup 1 ~x12 >= 1 ;  
rup 1 ~x11 1 ~x10 >= 1 ;  
rup 1 ~x11 >= 1 ;  
@obj soli x1 x2 x5 x8 ;  
rup 1 ~x8 1 ~x5 >= 1 ;  
rup 1 ~x8 >= 1 ;  
rup >= 1 ;  
output NONE ;  
conclusion BOUNDS 8 8 ;  
end pseudo-Boolean proof ;
```

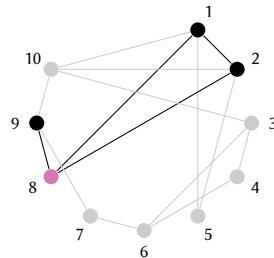


Backtrack from 8, 5
Only 4 vertices; can't have a ≥ 5 clique
Delete the edge

First Attempt at a Proof

pseudo-Boolean proof version 3.0

```
@obj soli x7 x9 x12 ;  
rup 1 ~x12 1 ~x7 >= 1 ;  
rup 1 ~x12 >= 1 ;  
rup 1 ~x11 1 ~x10 >= 1 ;  
rup 1 ~x11 >= 1 ;  
@obj soli x1 x2 x5 x8 ;  
rup 1 ~x8 1 ~x5 >= 1 ;  
rup 1 ~x8 >= 1 ;  
rup >= 1 ;  
output NONE ;  
conclusion BOUNDS 8 8 ;  
end pseudo-Boolean proof ;
```

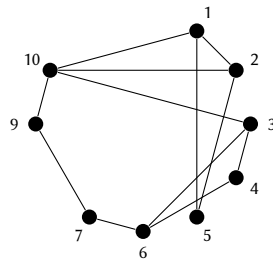


Backtrack from 8
Still not enough vertices
Delete the vertex

First Attempt at a Proof

pseudo-Boolean proof version 3.0

```
@obj soli x7 x9 x12 ;  
rup 1 ~x12 1 ~x7 >= 1 ;  
rup 1 ~x12 >= 1 ;  
rup 1 ~x11 1 ~x10 >= 1 ;  
rup 1 ~x11 >= 1 ;  
@obj soli x1 x2 x5 x8 ;  
rup 1 ~x8 1 ~x5 >= 1 ;  
rup 1 ~x8 >= 1 ;  
rup >= 1 ;  
output NONE ;  
conclusion BOUNDS 8 8 ;  
end pseudo-Boolean proof ;
```



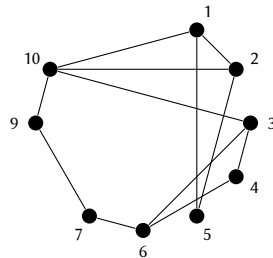
Remaining graph is 3-colourable
Backtrack from root node

First Attempt at a Proof

pseudo-Boolean proof version 3.0

```
@obj soli x7 x9 x12 ;  
rup 1 ~x12 1 ~x7 >= 1 ;  
rup 1 ~x12 >= 1 ;  
rup 1 ~x11 1 ~x10 >= 1 ;  
rup 1 ~x11 >= 1 ;  
@obj soli x1 x2 x5 x8 ;  
rup 1 ~x8 1 ~x5 >= 1 ;  
rup 1 ~x8 >= 1 ;  
rup >= 1 ;
```

```
output NONE ;  
conclusion BOUNDS 8 8 ;  
end pseudo-Boolean proof ;
```



Finish with what we've concluded
We specify a lower and an upper bound
Remember we're minimising the number of vertices we *don't* select, so a 4-clique has an objective value of $12 - 4 = 8$
("output" is for features we're not using)

Verifying This Proof (Or Not...)

```
$ veripb clique.opb clique-attempt-one.pbp
```

```
Running VeriPB version 3.0.2
```

```
Error: Checking error at clique-attempt-one.pbp:5
```

Caused by:

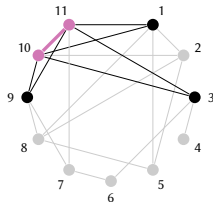
```
The constraint is not implied by reverse unit propagation  
(RUP) from core and derived database. Rerun with the option  
'--trace-failed' to see the propagations. . . .
```

Verifying This Proof (Or Not...)

```
$ veripb clique.opb clique-attempt-one.pbp
Running VeriPB version 3.0.2
Error: Checking error at clique-attempt-one.pbp:5
```

Caused by:

The constraint is not implied by reverse unit propagation (RUP) from core and derived database. Rerun with the option '--trace-failed' to see the propagations. . . .



Verifying This Proof (Or Not...)

```
$ veripb --trace clique.opb clique-attempt-one.pbp
Running VeriPB version 3.0.2
  Objective: min 1 ~x1 1 ~x2 . . . 1 ~x11 1 ~x12 0
  ConstraintId 1: 1 ~x1 1 ~x3 >= 1
  ConstraintId 2: 1 ~x2 1 ~x3 >= 1
  . . .
  ConstraintId 41: 1 ~x11 1 ~x12 >= 1
line   1: pseudo-Boolean proof version 3.0
line   2: @obj soli x7 x9 x12 ;
  ConstraintId 42: 1 x1 1 x2 . . . 1 x11 1 x12 >= 4
line   3: rup 1 ~x12 1 ~x7 >= 1 ;
  ConstraintId 43: 1 ~x7 1 ~x12 >= 1
line   4: rup 1 ~x12 >= 1 ;
  ConstraintId 44: 1 ~x12 >= 1
line   5: rup 1 ~x11 1 ~x10 >= 1 ;
Error: Checking error at clique-attempt-one.pbp:5
```

Verifying This Proof (Or Not...)

```
$ veripb --trace-failed clique.opb clique-attempt-one.pbp
Running VeriPB version 3.0.2
Propagation check failed! The propagation had the following trail:
  propagations in format: <assignment> (<reason constraint>)
    ~x12 (1 ~x12 >= 1)
    x10 (1 x10 1 x11 >= 2)
    x11 (1 x10 1 x11 >= 2)
    ~x4 (1 ~x4 1 ~x10 >= 1)
    ~x5 (1 ~x5 1 ~x10 >= 1)
    ~x6 (1 ~x6 1 ~x10 >= 1)
    ~x7 (1 ~x7 1 ~x10 >= 1)
    ~x8 (1 ~x8 1 ~x10 >= 1)
    ~x2 (1 ~x2 1 ~x11 >= 1)
Error: Checking error at clique-attempt-one.pbp:5
```

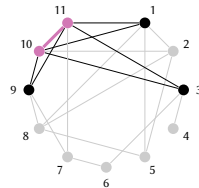
The Problem of Counting

The proof checker isn't smart enough to work out that

- vertices 1, 3 and 9 can all be given the same colour...
- and each colour class contributes at most one vertex to a clique...
- so “find me a clique with more than three vertices” can't be satisfied once we've accepted both 10 and 11

Unit propagation never sees this:

- The bound needs **counting**
- But the encoding only ever talks about **pairs** of vertices



Dealing with Colourings

So the colour bound doesn't follow by reverse unit propagation...

But we can lazily recover at-most-one constraints for each colour class!

Dealing with Colourings

So the colour bound doesn't follow by reverse unit propagation...

But we can lazily recover at-most-one constraints for each colour class!

$$\begin{aligned}
 &(\bar{x}_1 + \bar{x}_6 \geq 1) \\
 + &(\bar{x}_1 + \bar{x}_9 \geq 1) &&= 2\bar{x}_1 + \bar{x}_6 + \bar{x}_9 \geq 2 \\
 + &(\bar{x}_6 + \bar{x}_9 \geq 1) &&= 2\bar{x}_1 + 2\bar{x}_6 + 2\bar{x}_9 \geq 3 \\
 &&&/ 2 &&= \bar{x}_1 + \bar{x}_6 + \bar{x}_9 \geq 2 \\
 &&&&&\text{i.e. } x_1 + x_6 + x_9 \leq 1
 \end{aligned}$$

Dealing with Colourings

So the colour bound doesn't follow by reverse unit propagation...

But we can lazily recover at-most-one constraints for each colour class!

$$\begin{aligned}
 &(\bar{x}_1 + \bar{x}_6 \geq 1) \\
 + &(\bar{x}_1 + \bar{x}_9 \geq 1) &&= 2\bar{x}_1 + \bar{x}_6 + \bar{x}_9 \geq 2 \\
 + &(\bar{x}_6 + \bar{x}_9 \geq 1) &&= 2\bar{x}_1 + 2\bar{x}_6 + 2\bar{x}_9 \geq 3 \\
 &&&/ 2 &&= \bar{x}_1 + \bar{x}_6 + \bar{x}_9 \geq 2 \\
 &&&&&\text{i.e. } x_1 + x_6 + x_9 \leq 1
 \end{aligned}$$

This generalises to colour classes of any size v

- Each non-edge is used exactly once, $v(v-1)/2 - 1$ additions
- $v - 3$ multiplications and $v - 2$ divisions

Solvers don't need to "understand" cutting planes to write out this derivation

Details on At-Most-One Constraint Recovery (3 Variables)

- Derive constraint $x_1 + \dots + x_n \leq 1$ from $\binom{n}{2}$ constraints $x_i + x_j \leq 1$
- I.e., $\bar{x}_1 + \dots + \bar{x}_n \geq n - 1$ from constraints $\bar{x}_i + \bar{x}_j \geq 1$ in normalised form
- Need this to prove that **at most one vertex per colour class** can be a member of the clique

Details on At-Most-One Constraint Recovery (3 Variables)

- Derive constraint $x_1 + \dots + x_n \leq 1$ from $\binom{n}{2}$ constraints $x_i + x_j \leq 1$
- I.e., $\bar{x}_1 + \dots + \bar{x}_n \geq n - 1$ from constraints $\bar{x}_i + \bar{x}_j \geq 1$ in normalised form
- Need this to prove that **at most one vertex per colour class** can be a member of the clique

$$\begin{array}{r}
 \text{Add} \quad \bar{x}_1 + \bar{x}_2 \geq 1 \quad \bar{x}_1 + \bar{x}_3 \geq 1 \\
 \hline
 2\bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2
 \end{array}$$

Details on At-Most-One Constraint Recovery (3 Variables)

- Derive constraint $x_1 + \dots + x_n \leq 1$ from $\binom{n}{2}$ constraints $x_i + x_j \leq 1$
- I.e., $\bar{x}_1 + \dots + \bar{x}_n \geq n - 1$ from constraints $\bar{x}_i + \bar{x}_j \geq 1$ in normalised form
- Need this to prove that **at most one vertex per colour class** can be a member of the clique

$$\begin{array}{rcl}
 & \bar{x}_1 + \bar{x}_2 \geq 1 & \bar{x}_1 + \bar{x}_3 \geq 1 \\
 \text{Add} & \hline
 & 2\bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2 & \bar{x}_2 + \bar{x}_3 \geq 1 \\
 \text{Add} & \hline
 & 2\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 \geq 3
 \end{array}$$

Details on At-Most-One Constraint Recovery (3 Variables)

- Derive constraint $x_1 + \dots + x_n \leq 1$ from $\binom{n}{2}$ constraints $x_i + x_j \leq 1$
- I.e., $\bar{x}_1 + \dots + \bar{x}_n \geq n - 1$ from constraints $\bar{x}_i + \bar{x}_j \geq 1$ in normalised form
- Need this to prove that **at most one vertex per colour class** can be a member of the clique

$$\begin{array}{rcl}
 & \bar{x}_1 + \bar{x}_2 \geq 1 & \bar{x}_1 + \bar{x}_3 \geq 1 \\
 \text{Add} & \hline
 & 2\bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2 & \bar{x}_2 + \bar{x}_3 \geq 1 \\
 \text{Add} & \hline
 & 2\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 \geq 3 \\
 \text{Divide by 2} & \hline
 & \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq \frac{3}{2}
 \end{array}$$

Details on At-Most-One Constraint Recovery (3 Variables)

- Derive constraint $x_1 + \dots + x_n \leq 1$ from $\binom{n}{2}$ constraints $x_i + x_j \leq 1$
- I.e., $\bar{x}_1 + \dots + \bar{x}_n \geq n - 1$ from constraints $\bar{x}_i + \bar{x}_j \geq 1$ in normalised form
- Need this to prove that **at most one vertex per colour class** can be a member of the clique

$$\begin{array}{rcl}
 & \bar{x}_1 + \bar{x}_2 \geq 1 & \bar{x}_1 + \bar{x}_3 \geq 1 \\
 \text{Add} & \hline
 & 2\bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2 & \bar{x}_2 + \bar{x}_3 \geq 1 \\
 \text{Add} & \hline
 & 2\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 \geq 3 \\
 \text{Divide by 2} & \hline
 & \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2
 \end{array}$$

Details on At-Most-One Constraint Recovery (3 Variables)

- Derive constraint $x_1 + \dots + x_n \leq 1$ from $\binom{n}{2}$ constraints $x_i + x_j \leq 1$
- I.e., $\bar{x}_1 + \dots + \bar{x}_n \geq n - 1$ from constraints $\bar{x}_i + \bar{x}_j \geq 1$ in normalised form
- Need this to prove that **at most one vertex per colour class** can be a member of the clique

$$\begin{array}{rcl}
 & \bar{x}_1 + \bar{x}_2 \geq 1 & \bar{x}_1 + \bar{x}_3 \geq 1 \\
 \text{Add} & \hline
 & 2\bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2 & \bar{x}_2 + \bar{x}_3 \geq 1 \\
 \text{Add} & \hline
 & 2\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 \geq 3 \\
 \text{Divide by 2} & \hline
 & \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2
 \end{array}$$

Assuming constraint $\bar{x}_i + \bar{x}_j \geq 1$ has label @C*ij*, get VERIPB code

```
pol @C12 @C13 + @C23 + 2 d ;
```

Details on At-Most-One Constraint Recovery (4 and More Variables)

- Derive constraint $x_1 + \dots + x_n \leq 1$ from $\binom{n}{2}$ constraints $x_i + x_j \leq 1$
- I.e., $\bar{x}_1 + \dots + \bar{x}_n \geq n - 1$ from constraints $\bar{x}_i + \bar{x}_j \geq 1$ in normalised form
- Need this to prove that **at most one vertex per colour class** can be a member of the clique

$$\bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2$$

Details on At-Most-One Constraint Recovery (4 and More Variables)

- Derive constraint $x_1 + \dots + x_n \leq 1$ from $\binom{n}{2}$ constraints $x_i + x_j \leq 1$
- I.e., $\bar{x}_1 + \dots + \bar{x}_n \geq n - 1$ from constraints $\bar{x}_i + \bar{x}_j \geq 1$ in normalised form
- Need this to prove that **at most one vertex per colour class** can be a member of the clique

$$\text{Multiply by 2} \quad \frac{\bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2}{2\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 \geq 4}$$

Details on At-Most-One Constraint Recovery (4 and More Variables)

- Derive constraint $x_1 + \dots + x_n \leq 1$ from $\binom{n}{2}$ constraints $x_i + x_j \leq 1$
- I.e., $\bar{x}_1 + \dots + \bar{x}_n \geq n - 1$ from constraints $\bar{x}_i + \bar{x}_j \geq 1$ in normalised form
- Need this to prove that **at most one vertex per colour class** can be a member of the clique

$$\begin{array}{rcl}
 \text{Multiply by 2} & \frac{\bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2}{2\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 \geq 4} & \bar{x}_1 + \bar{x}_4 \geq 1 \\
 \text{Add} & \frac{2\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 \geq 4}{3\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 + \bar{x}_4 \geq 5} & \bar{x}_2 + \bar{x}_4 \geq 1 \\
 \text{Add} & \frac{3\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 + \bar{x}_4 \geq 5}{3\bar{x}_1 + 3\bar{x}_2 + 2\bar{x}_3 + 2\bar{x}_4 \geq 6} & \bar{x}_3 + \bar{x}_4 \geq 1 \\
 \text{Add} & \frac{3\bar{x}_1 + 3\bar{x}_2 + 2\bar{x}_3 + 2\bar{x}_4 \geq 6}{3\bar{x}_1 + 3\bar{x}_2 + 3\bar{x}_3 + 3\bar{x}_4 \geq 7} &
 \end{array}$$

Details on At-Most-One Constraint Recovery (4 and More Variables)

- Derive constraint $x_1 + \dots + x_n \leq 1$ from $\binom{n}{2}$ constraints $x_i + x_j \leq 1$
- I.e., $\bar{x}_1 + \dots + \bar{x}_n \geq n - 1$ from constraints $\bar{x}_i + \bar{x}_j \geq 1$ in normalised form
- Need this to prove that **at most one vertex per colour class** can be a member of the clique

$$\begin{array}{rcl}
 \text{Multiply by 2} & \frac{\bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2}{2\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 \geq 4} & \bar{x}_1 + \bar{x}_4 \geq 1 \\
 \text{Add} & \frac{2\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 \geq 4}{3\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 + \bar{x}_4 \geq 5} & \bar{x}_2 + \bar{x}_4 \geq 1 \\
 \text{Add} & \frac{3\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 + \bar{x}_4 \geq 5}{3\bar{x}_1 + 3\bar{x}_2 + 2\bar{x}_3 + 2\bar{x}_4 \geq 6} & \bar{x}_3 + \bar{x}_4 \geq 1 \\
 \text{Add} & \frac{3\bar{x}_1 + 3\bar{x}_2 + 2\bar{x}_3 + 2\bar{x}_4 \geq 6}{3\bar{x}_1 + 3\bar{x}_2 + 3\bar{x}_3 + 3\bar{x}_4 \geq 7} & \\
 \text{Divide by 3} & \frac{3\bar{x}_1 + 3\bar{x}_2 + 3\bar{x}_3 + 3\bar{x}_4 \geq 7}{\bar{x}_1 + \bar{x}_2 + \bar{x}_3 + \bar{x}_4 \geq \left\lceil \frac{7}{3} \right\rceil} &
 \end{array}$$

Details on At-Most-One Constraint Recovery (4 and More Variables)

- Derive constraint $x_1 + \dots + x_n \leq 1$ from $\binom{n}{2}$ constraints $x_i + x_j \leq 1$
- I.e., $\bar{x}_1 + \dots + \bar{x}_n \geq n - 1$ from constraints $\bar{x}_i + \bar{x}_j \geq 1$ in normalised form
- Need this to prove that **at most one vertex per colour class** can be a member of the clique

$$\begin{array}{rcl}
 \text{Multiply by 2} & \frac{\bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2}{2\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 \geq 4} & \bar{x}_1 + \bar{x}_4 \geq 1 \\
 \text{Add} & \frac{2\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 \geq 4}{3\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 + \bar{x}_4 \geq 5} & \bar{x}_2 + \bar{x}_4 \geq 1 \\
 \text{Add} & \frac{3\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 + \bar{x}_4 \geq 5}{3\bar{x}_1 + 3\bar{x}_2 + 2\bar{x}_3 + 2\bar{x}_4 \geq 6} & \bar{x}_3 + \bar{x}_4 \geq 1 \\
 \text{Add} & \frac{3\bar{x}_1 + 3\bar{x}_2 + 2\bar{x}_3 + 2\bar{x}_4 \geq 6}{3\bar{x}_1 + 3\bar{x}_2 + 3\bar{x}_3 + 3\bar{x}_4 \geq 7} & \\
 \text{Divide by 3} & \frac{3\bar{x}_1 + 3\bar{x}_2 + 3\bar{x}_3 + 3\bar{x}_4 \geq 7}{\bar{x}_1 + \bar{x}_2 + \bar{x}_3 + \bar{x}_4 \geq 3} &
 \end{array}$$

Details on At-Most-One Constraint Recovery (4 and More Variables)

- Derive constraint $x_1 + \dots + x_n \leq 1$ from $\binom{n}{2}$ constraints $x_i + x_j \leq 1$
- I.e., $\bar{x}_1 + \dots + \bar{x}_n \geq n - 1$ from constraints $\bar{x}_i + \bar{x}_j \geq 1$ in normalised form
- Need this to prove that **at most one vertex per colour class** can be a member of the clique

$$\begin{array}{rcl}
 \text{Multiply by 2} & \frac{\bar{x}_1 + \bar{x}_2 + \bar{x}_3 \geq 2}{2\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 \geq 4} & \bar{x}_1 + \bar{x}_4 \geq 1 \\
 \text{Add} & \frac{2\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 \geq 4}{3\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 + \bar{x}_4 \geq 5} & \bar{x}_2 + \bar{x}_4 \geq 1 \\
 \text{Add} & \frac{3\bar{x}_1 + 2\bar{x}_2 + 2\bar{x}_3 + \bar{x}_4 \geq 5}{3\bar{x}_1 + 3\bar{x}_2 + 2\bar{x}_3 + 2\bar{x}_4 \geq 6} & \bar{x}_3 + \bar{x}_4 \geq 1 \\
 \text{Add} & \frac{3\bar{x}_1 + 3\bar{x}_2 + 2\bar{x}_3 + 2\bar{x}_4 \geq 6}{3\bar{x}_1 + 3\bar{x}_2 + 3\bar{x}_3 + 3\bar{x}_4 \geq 7} & \\
 \text{Divide by 3} & \frac{3\bar{x}_1 + 3\bar{x}_2 + 3\bar{x}_3 + 3\bar{x}_4 \geq 7}{\bar{x}_1 + \bar{x}_2 + \bar{x}_3 + \bar{x}_4 \geq 3} &
 \end{array}$$

Assuming constraint $\bar{x}_i + \bar{x}_j \geq 1$ has label @C $_{ij}$, VERIPB code for full derivation is

```
pol @C12 @C13 + @C23 + 2 d 2 * @C14 + @C24 + @C34 + 3 d ;
```

Details on At-Most-One Constraint Recovery (4 and More Variables)

- Derive constraint $x_1 + \dots + x_n \leq 1$ from $\binom{n}{2}$ constraints $x_i + x_j \leq 1$
- I.e., $\bar{x}_1 + \dots + \bar{x}_n \geq n - 1$ from constraints $\bar{x}_i + \bar{x}_j \geq 1$ in normalised form
- Need this to prove that **at most one vertex per colour class** can be a member of the clique

$$\begin{array}{rcl}
 \overline{x}_1 + \overline{x}_2 + \overline{x}_3 & \geq & 2 \\
 \text{Multiply by 2} & & \\
 \hline
 2\overline{x}_1 + 2\overline{x}_2 + 2\overline{x}_3 & \geq & 4 \qquad \overline{x}_1 + \overline{x}_4 \geq 1 \\
 \text{Add} & & \\
 \hline
 3\overline{x}_1 + 2\overline{x}_2 + 2\overline{x}_3 + \overline{x}_4 & \geq & 5 \qquad \overline{x}_2 + \overline{x}_4 \geq 1 \\
 \text{Add} & & \\
 \hline
 3\overline{x}_1 + 3\overline{x}_2 + 2\overline{x}_3 + 2\overline{x}_4 & \geq & 6 \qquad \overline{x}_3 + \overline{x}_4 \geq 1 \\
 \text{Add} & & \\
 \hline
 3\overline{x}_1 + 3\overline{x}_2 + 3\overline{x}_3 + 3\overline{x}_4 & \geq & 7 \\
 \text{Divide by 3} & & \\
 \hline
 \overline{x}_1 + \overline{x}_2 + \overline{x}_3 + \overline{x}_4 & \geq & \frac{7}{3}
 \end{array}$$

Assuming constraint $\bar{x}_i + \bar{x}_j \geq 1$ has label @Cij, VERIPB code for full derivation is

$$\text{pol} \quad @C12 \quad @C13 \quad + \quad @C23 \quad + \quad 2 \quad d \quad 2 \quad * \quad @C14 \quad + \quad @C24 \quad + \quad @C34 \quad + \quad 3 \quad d \quad ;$$

(and this pattern generalizes to any number of variables)

What the Revised Proof with Explicit Counting Looks Like

pseudo-Boolean proof version 3.0

```
@obj soli x7 x9 x12 ;
rup 1 ~x12 1 ~x7 >= 1 ;
pol @noedge1_6 @noedge1_9 + @noedge6_9 + 2 d @obj + ;
rup 1 ~x12 >= 1 ;
pol @noedge1_3 @noedge1_9 + @noedge3_9 + 2 d @obj + ;
rup 1 ~x11 1 ~x10 >= 1 ;
pol @noedge1_3 @noedge1_7 + @noedge3_7 + 2 d @obj + ;
rup 1 ~x11 >= 1 ;
@obj soli x1 x2 x5 x8 ;
rup 1 ~x8 1 ~x5 >= 1 ;
pol @obj @noedge1_9 + ;
rup 1 ~x8 >= 1 ;
pol @noedge1_3 @noedge1_7 + @noedge3_7 + 2 d @noedge2_4 @noedge2_9 + @noedge4_9 + 2 d +
    @noedge5_6 @noedge5_10 + @noedge6_10 + 2 d + @obj + ;
rup >= 1 ;
output NONE ;
conclusion BOUNDS 8 8 ;
end pseudo-Boolean proof ;
```

What the Revised Proof with Explicit Counting Looks Like

pseudo-Boolean proof version 3.0

@obj soli x7 x9 x12 ;

rup 1 ~x12 1 ~x7 >= 1 ;

pol @noedge1_6 @noedge1_9 + @noedge6_9 + 2 d @obj + ;

rup 1 ~x12 >= 1 ;

pol @noedge1_3 @noedge1_9 + @noedge3_9 + 2 d @obj + ;

rup 1 ~x11 1 ~x10 >= 1 ;

pol @noedge1_3 @noedge1_7 + @noedge3_7 + 2 d @obj + ;

rup 1 ~x11 >= 1 ;

@obj soli x1 x2 x5 x8 ;

rup 1 ~x8 1 ~x5 >= 1 ;

pol @obj @noedge1_9 + ;

rup 1 ~x8 >= 1 ;

pol @noedge1_3 @noedge1_7 + @noedge3_7 + 2 d @noedge2_4 @noedge2_9 + @noedge4_9 + 2 d +

@noedge5_6 @noedge5_10 + @noedge6_10 + 2 d + @obj + ;

rup >= 1 ;

output NONE ;

conclusion BOUNDS 8 8 ;

end pseudo-Boolean proof ;

Failing RUP steps have become **pol** derivations: add three non-adjacency constraints together, divide by two, then add the result to the solution-improving constraint

The labels come straight out of the OPB file

What the Revised Proof with Explicit Counting Looks Like

pseudo-Boolean proof version 3.0

```
@obj soli x7 x9 x12 ;
rup 1 ~x12 1 ~x7 >= 1 ;
pol @noedge1_6 @noedge1_9 + @noedge6_9 + 2 d @obj + ;
rup 1 ~x12 >= 1 ;
pol @noedge1_3 @noedge1_9 + @noedge3_9 + 2 d @obj + ;
rup 1 ~x11 1 ~x10 >= 1 ;
pol @noedge1_3 @noedge1_7 + @noedge3_7 + 2 d @obj + ;
rup 1 ~x11 >= 1 ;
@obj soli x1 x2 x5 x8 ;
rup 1 ~x8 1 ~x5 >= 1 ;
pol @obj @noedge1_9 + ;
rup 1 ~x8 >= 1 ;
```

```
pol @noedge1_3 @noedge1_7 + @noedge3_7 + 2 d @noedge2_4 @noedge2_9 + @noedge4_9 + 2 d +
  @noedge5_6 @noedge5_10 + @noedge6_10 + 2 d + @obj + ;
```

```
rup >= 1 ;
output NONE ;
conclusion BOUNDS 8 8 ;
end pseudo-Boolean proof ;
```

Three colour classes here, so three at-most-one constraints, all added to the solution-improving constraint

Verifying This Proof (For Real, This Time)

```
$ veripb clique.opb clique-new-attempt.pbp
Running VeriPB version 3.0.2
s VERIFIED BOUNDS 8 <= obj <= 8
```

No errors this time — every step is justified

Note that the checker **knows nothing** about

- cliques,
- colourings, or
- bound functions

Verifying This Proof (For Real, This Time)

```
$ veripb --trace clique.opb clique-new-attempt.pbp
Running VeriPB version 3.0.2
Objective: min 1 ~x1 1 ~x2 . . . 1 ~x11 1 ~x12 0
ConstraintId 1: 1 ~x1 1 ~x3 >= 1
. . .
ConstraintId 41: 1 ~x11 1 ~x12 >= 1
line 2: @obj soli x7 x9 x12 ;
ConstraintId 42: 1 x1 1 x2 . . . 1 x11 1 x12 >= 4
line 3: rup 1 ~x12 1 ~x7 >= 1 ;
ConstraintId 43: 1 ~x7 1 ~x12 >= 1
line 4: pol @noedge1_6 @noedge1_9 + @noedge6_9 + 2 d @obj + ;
ConstraintId 44: 1 x2 1 x3 1 x4 1 x5 1 x7 1 x8 1 x10 1 x11 1 x12 >= 3
. . .
line 16: rup >= 1 ;
ConstraintId 55: >= 1
s VERIFIED BOUNDS 8 <= obj <= 8
```

Why a General-Purpose Proof System?

Why not just put a “colouring bound” rule into the proof format?

Why a General-Purpose Proof System?

Why not just put a “colouring bound” rule into the proof format?

- There are dozens of different colouring algorithms

Why a General-Purpose Proof System?

Why not just put a “colouring bound” rule into the proof format?

- There are dozens of different colouring algorithms
- And extensions of these algorithms using MaxSAT reasoning

Why a General-Purpose Proof System?

Why not just put a “colouring bound” rule into the proof format?

- There are dozens of different colouring algorithms
- And extensions of these algorithms using MaxSAT reasoning
- Even more for [maximum weight clique](#), where weighted colour class rules get extremely clever (vertices can split their weight between several colours)

Why a General-Purpose Proof System?

Why not just put a “colouring bound” rule into the proof format?

- There are dozens of different colouring algorithms
- And extensions of these algorithms using MaxSAT reasoning
- Even more for [maximum weight clique](#), where weighted colour class rules get extremely clever (vertices can split their weight between several colours)
- And we would need to do the same again for every other kind of solver

Why a General-Purpose Proof System?

Why not just put a “colouring bound” rule into the proof format?

- There are dozens of different colouring algorithms
- And extensions of these algorithms using MaxSAT reasoning
- Even more for [maximum weight clique](#), where weighted colour class rules get extremely clever (vertices can split their weight between several colours)
- And we would need to do the same again for every other kind of solver

We would rather not have a proof format and a proof checker per solver, per paper...

Why a General-Purpose Proof System?

Why not just put a “colouring bound” rule into the proof format?

- There are dozens of different colouring algorithms
- And extensions of these algorithms using MaxSAT reasoning
- Even more for [maximum weight clique](#), where weighted colour class rules get extremely clever (vertices can split their weight between several colours)
- And we would need to do the same again for every other kind of solver

We would rather not have a proof format and a proof checker per solver, per paper...

Cutting planes proof system to the rescue!

- Turns out to be strong enough for all of these cases
- Proof length scales linearly with solver reasoning time (asymptotically)

Different Clique Algorithms

Different search orders?

- ✓ Irrelevant for proof logging

Using local search to initialise?

- ✓ Just log the incumbent

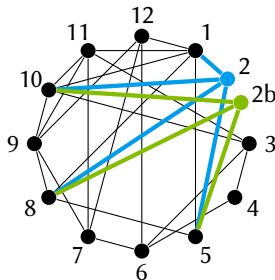
Different bound functions?

- Is cutting planes strong enough to justify every useful bound function ever invented?
- So far, seems like it...

Weighted cliques?

- ✓ Multiply a colour class by its largest weight
- ✓ Also works for vertices “split between colour classes”

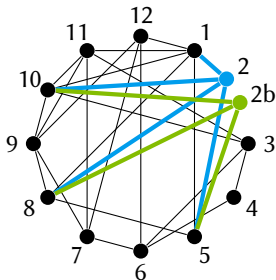
Lazy Global Domination for Maximum Clique [MP16]



Can ignore vertex 2b

- Every neighbour of 2b is also a neighbour of 2
- This is an example of **dominance** reasoning

Lazy Global Domination for Maximum Clique [MP16]



Can ignore vertex 2b

- Every neighbour of 2b is also a neighbour of 2
- This is an example of **dominance** reasoning

Dominance-based strengthening rule can justify this

- Even when detected dynamically during search

Reducing the Trust Base

We are still trusting rather a lot

Reducing the Trust Base

We are still trusting rather a lot

- That the proof checker is correct

Reducing the Trust Base

We are still trusting rather a lot

- That the proof checker is correct
- That the pseudo-Boolean encoding really does describe the clique problem we were given

Reducing the Trust Base

We are still trusting rather a lot

- That the proof checker is correct
- That the pseudo-Boolean encoding really does describe the clique problem we were given

Both can be dealt with:

Reducing the Trust Base

We are still trusting rather a lot

- That the proof checker is correct
- That the pseudo-Boolean encoding really does describe the clique problem we were given

Both can be dealt with:

- VERIPB can **elaborate** a proof into a simpler **kernel format**

Reducing the Trust Base

We are still trusting rather a lot

- That the proof checker is correct
- That the pseudo-Boolean encoding really does describe the clique problem we were given

Both can be dealt with:

- VERIPB can **elaborate** a proof into a simpler **kernel format**
- Uses only rules that the **formally verified** checker **CAKEPB** knows about
- The proof checker CAKEPB (which is written in CAKEML and proved correct in HOL4)
 - reads the **original graph file**
 - does a **formally verified translation to 0-1 ILP**
 - performs **formally verified checking** of the kernel proof against this translation

Reducing the Trust Base

We are still trusting rather a lot

- That the proof checker is correct
- That the pseudo-Boolean encoding really does describe the clique problem we were given

Both can be dealt with:

- VERIPB can **elaborate** a proof into a simpler **kernel format**
- Uses only rules that the **formally verified** checker **CAKEPB** knows about
- The proof checker CAKEPB (which is written in CAKEML and proved correct in HOL4)
 - reads the **original graph file**
 - does a **formally verified translation to 0–1 ILP**
 - performs **formally verified checking** of the kernel proof against this translation
- So the answer is formally verified in terms of the original input

End-to-End Verification for Maximum Clique

```
$ glasgow_clique_solver brock200_4.clq
recursions = 56613
clique = 17 (12 19 28 29 38 54 65 71 79 93 117 127 139 161 165 186 192)
runtime = 0.02153s

$ glasgow_clique_solver brock200_4.clq --prove brock200_4
runtime = 0.266475s

$ cake_pb_clique brock200_4.clq > brock200_4.verifiedopb

$ veripb --elaborate brock200_4.corepb brock200_4.verifiedopb brock200_4.pbp
s VERIFIED BOUNDS 183 <= obj <= 183

$ cake_pb_clique brock200_4.clq brock200_4.corepb
s VERIFIED MAX CLIQUE SIZE |CLIQUE| = 17
```


What's Left to Trust?

Still have to trust:

- The HOL4 theorem prover
- That the formal model of the CAKEML environment matches the hardware it runs on
- The HOL4 definition of what it means to be a maximum clique
- Input parsing and output formatting

What's Left to Trust?

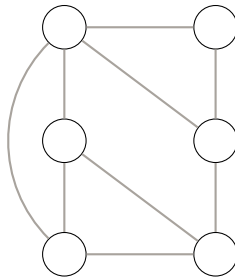
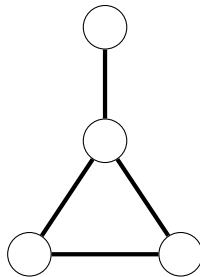
Still have to trust:

- The HOL4 theorem prover
- That the formal model of the CAKEML environment matches the hardware it runs on
- The HOL4 definition of what it means to be a maximum clique
- Input parsing and output formatting

No need to trust, or even to know about:

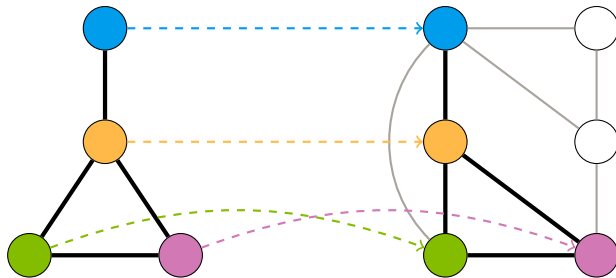
- How the solver works
- That the VERIPB proof system is sound
- Or even what “pseudo-Boolean” means

Subgraph Isomorphism



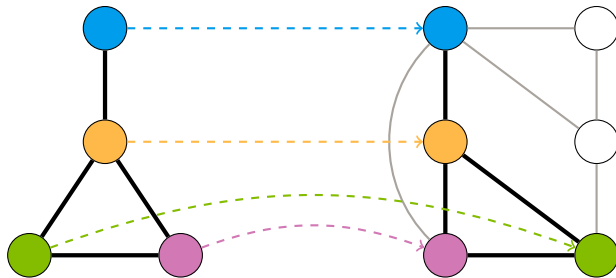
- Find the **pattern** inside the **target**
- Applications in compilers, biochemistry, model checking, pattern recognition, ...
- Often want to find **all** matches

Subgraph Isomorphism



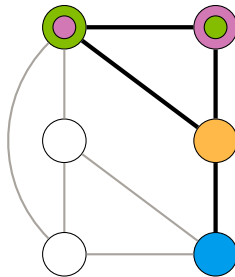
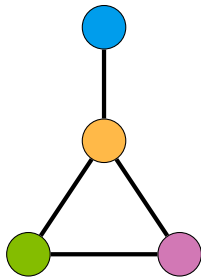
- Find the **pattern** inside the **target**
- Applications in compilers, biochemistry, model checking, pattern recognition, ...
- Often want to find **all** matches

Subgraph Isomorphism



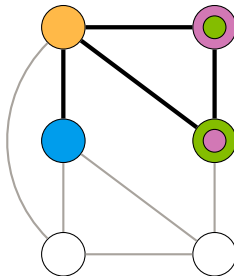
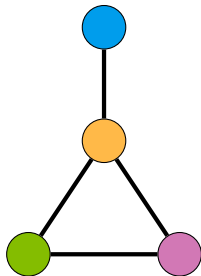
- Find the **pattern** inside the **target**
- Applications in compilers, biochemistry, model checking, pattern recognition, ...
- Often want to find **all** matches

Subgraph Isomorphism



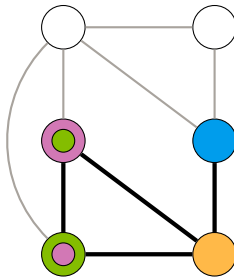
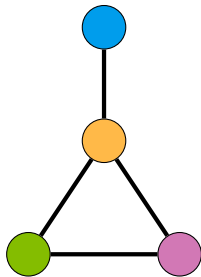
- Find the **pattern** inside the **target**
- Applications in compilers, biochemistry, model checking, pattern recognition, ...
- Often want to find **all** matches

Subgraph Isomorphism



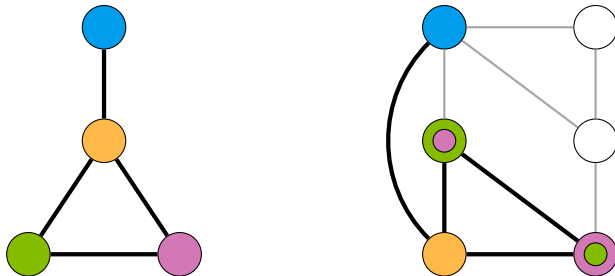
- Find the **pattern** inside the **target**
- Applications in compilers, biochemistry, model checking, pattern recognition, ...
- Often want to find **all** matches

Subgraph Isomorphism



- Find the **pattern** inside the **target**
- Applications in compilers, biochemistry, model checking, pattern recognition, ...
- Often want to find **all** matches

Subgraph Isomorphism



- Find the **pattern** inside the **target**
- Applications in compilers, biochemistry, model checking, pattern recognition, ...
- Often want to find **all** matches

Subgraph Isomorphism in Pseudo-Boolean Form

Each pattern vertex gets a target vertex:

$$\sum_{t \in V(T)} x_{p,t} = 1 \qquad p \in V(P)$$

Subgraph Isomorphism in Pseudo-Boolean Form

Each pattern vertex gets a target vertex:

$$\sum_{t \in V(T)} x_{p,t} = 1 \qquad p \in V(P)$$

Each target vertex may be used at most once:

$$\sum_{p \in V(P)} -x_{p,t} \geq -1 \qquad t \in V(T)$$

Subgraph Isomorphism in Pseudo-Boolean Form

Each pattern vertex gets a target vertex:

$$\sum_{t \in V(T)} x_{p,t} = 1 \quad p \in V(P)$$

Each target vertex may be used at most once:

$$\sum_{p \in V(P)} -x_{p,t} \geq -1 \quad t \in V(T)$$

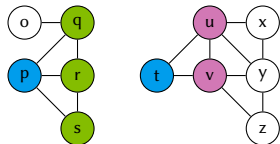
Adjacency constraints: if p maps to t , then neighbours of p map to neighbours of t :

$$\bar{x}_{p,t} + \sum_{u \in N(t)} x_{q,u} \geq 1 \quad p \in V(P), q \in N(p), t \in V(T)$$

Observe $N(p) = \{q, r, s\}$ and $N(t) = \{u, v\}$

We wish to derive $\bar{x}_{p,t} \geq 1$

Degree Reasoning in Cutting Planes



Adjacency:

$$\bar{x}_{p,t} + x_{q,u} + x_{q,v} \geq 1$$

$$\bar{x}_{p,t} + x_{r,u} + x_{r,v} \geq 1$$

$$\bar{x}_{p,t} + x_{s,u} + x_{s,v} \geq 1$$

Injectivity:

$$-x_{o,u} + -x_{p,u} + -x_{q,u} + -x_{r,u} + -x_{s,u} \geq -1$$

$$-x_{o,v} + -x_{p,v} + -x_{q,v} + -x_{r,v} + -x_{s,v} \geq -1$$

Literal axioms:

$$x_{o,u} \geq 0$$

$$x_{o,v} \geq 0$$

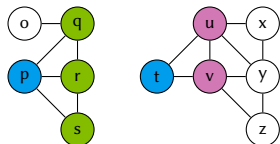
$$x_{p,u} \geq 0$$

$$x_{p,v} \geq 0$$

Add these together ...

$$3 \cdot \bar{x}_{p,t} \geq 1$$

Degree Reasoning in Cutting Planes



Adjacency:

$$\bar{x}_{p,t} + x_{q,u} + x_{q,v} \geq 1$$

$$\bar{x}_{p,t} + x_{r,u} + x_{r,v} \geq 1$$

$$\bar{x}_{p,t} + x_{s,u} + x_{s,v} \geq 1$$

Injectivity:

$$-x_{o,u} + -x_{p,u} + -x_{q,u} + -x_{r,u} + -x_{s,u} \geq -1$$

$$-x_{o,v} + -x_{p,v} + -x_{q,v} + -x_{r,v} + -x_{s,v} \geq -1$$

Literal axioms:

$$x_{o,u} \geq 0$$

$$x_{o,v} \geq 0$$

$$x_{p,u} \geq 0$$

$$x_{p,v} \geq 0$$

Add these together and divide by 3 to get

$$\bar{x}_{p,t} \geq 1$$

Degree Reasoning in VERIPB

```
pol @adj_p_t_q @adj_p_t_r + @adj_p_t_s + % sum adjacency constraints
    @inj_u + @inj_v +                     % sum injectivity constraints
    xo_u + xo_v +                         % cancel stray xo_* >= 0
    xp_u + xp_v +                         % cancel stray xp_* >= 0
    3 d ;                                % divide, and we're done
```

Degree Reasoning in VERIPB

```
pol @adj_p_t_q @adj_p_t_r + @adj_p_t_s +      % sum adjacency constraints
    @inj_u + @inj_v +                          % sum injectivity constraints
    xo_u + xo_v +                              % cancel stray xo_* >= 0
    xp_u + xp_v +                              % cancel stray xp_* >= 0
    3 d ;                                       % divide, and we're done
```

The labels come from the OPB file, so solver doesn't need to keep track of constraint IDs

Possible to certify (almost) all of the other things state-of-the-art subgraph solvers do:

- Conditional injectivity reasoning and filtering
- Distance filtering
- Neighbourhood degree sequences
- Path filtering
- Supplemental graphs

- ## ■ Supplemental graphs

Proof Logging for Other Forms of Reasoning

Possible to certify (almost) all of the other things state-of-the-art subgraph solvers do:

- Conditional injectivity reasoning and filtering
- Distance filtering
- Neighbourhood degree sequences
- Path filtering
- Supplemental graphs

Proof steps are “efficient” using cutting planes

- Length of proof scales linearly with time complexity of the reasoning algorithms
- Most proof steps require only trivial additional computations

Code for the certifying *Glasgow Subgraph Solver* at

<https://github.com/ciaranm/glasgow-subgraph-solver>

Limitations

Why trust the encoding?

- ✓ For clique and subgraph isomorphism this is now taken care of [GMM⁺24]
 - Has to be done again for each new problem, but not too bad

Limitations

Why trust the encoding?

- ✓ For clique and subgraph isomorphism this is now taken care of [GMM⁺24]
 - Has to be done again for each new problem, but not too bad

Proof logging can introduce large slowdowns

- Writing to disk is much slower than bit-parallel algorithms

What About Constraint Programming?

Non-Boolean variables?

Constraints?

- Encoding constraints in pseudo-Boolean form?
- Justifying inferences?

Reformulations?

Integer Variables in Constraint Programming (1/2)

How to deal with integer variables in constraint programming?

Given $A \in \{-3, \dots, 9\}$, the direct encoding is:

$$\begin{aligned} a_{=-3} + a_{=-2} + a_{=-1} + a_{=0} + a_{=1} + a_{=2} + a_{=3} \\ + a_{=4} + a_{=5} + a_{=6} + a_{=7} + a_{=8} + a_{=9} = 1 \end{aligned}$$

Integer Variables in Constraint Programming (1/2)

How to deal with integer variables in constraint programming?

Given $A \in \{-3, \dots, 9\}$, the direct encoding is:

$$a_{=-3} + a_{=-2} + a_{=-1} + a_{=0} + a_{=1} + a_{=2} + a_{=3} \\ + a_{=4} + a_{=5} + a_{=6} + a_{=7} + a_{=8} + a_{=9} = 1$$

This doesn't work for large domains...

We could use a binary encoding:

$$\begin{aligned} -16a_{\text{neg}} + 1a_{\text{b0}} + 2a_{\text{b1}} + 4a_{\text{b2}} + 8a_{\text{b3}} &\geq -3 && \text{and} \\ 16a_{\text{neg}} + -1a_{\text{b0}} + -2a_{\text{b1}} + -4a_{\text{b2}} + -8a_{\text{b3}} &\geq -9 \end{aligned}$$

Bad properties for solver propagation, but that isn't a problem for proof logging

Integer Variables in Constraint Programming (1/2)

How to deal with integer variables in constraint programming?

Given $A \in \{-3, \dots, 9\}$, the direct encoding is:

$$a_{=-3} + a_{=-2} + a_{=-1} + a_{=0} + a_{=1} + a_{=2} + a_{=3} \\ + a_{=4} + a_{=5} + a_{=6} + a_{=7} + a_{=8} + a_{=9} = 1$$

This doesn't work for large domains...

We could use a binary encoding:

$$\begin{aligned} -16a_{\text{neg}} + 1a_{b0} + 2a_{b1} + 4a_{b2} + 8a_{b3} &\geq -3 && \text{and} \\ 16a_{\text{neg}} + -1a_{b0} + -2a_{b1} + -4a_{b2} + -8a_{b3} &\geq -9 \end{aligned}$$

Bad properties for solver propagation, but that isn't a problem for proof logging

Convention in what follows:

- Upper-case A, B, C are **CP variables** used by solver
- Lower-case a, b, c are **corresponding Boolean variables** used for PB encoding in proof log

Integer Variables in Constraint Programming (2/2)

We can mix binary and order encodings! Define big-M linear inequalities encoding

$$a_{\geq 4} \Leftrightarrow -16a_{\text{neg}} + 1a_{b0} + 2a_{b1} + 4a_{b2} + 8a_{b3} \geq 4$$

$$a_{\geq 5} \Leftrightarrow -16a_{\text{neg}} + 1a_{b0} + 2a_{b1} + 4a_{b2} + 8a_{b3} \geq 5$$

$$a_{=4} \Leftrightarrow a_{\geq 4} \wedge \bar{a}_{\geq 5}$$

Integer Variables in Constraint Programming (2/2)

We can mix binary and order encodings! Define big-M linear inequalities encoding

$$a_{\geq 4} \Leftrightarrow -16a_{\text{neg}} + 1a_{b0} + 2a_{b1} + 4a_{b2} + 8a_{b3} \geq 4$$

$$a_{\geq 5} \Leftrightarrow -16a_{\text{neg}} + 1a_{b0} + 2a_{b1} + 4a_{b2} + 8a_{b3} \geq 5$$

$$a_{=4} \Leftrightarrow a_{\geq 4} \wedge \bar{a}_{\geq 5}$$

When creating $a_{\geq i}$, also introduce pseudo-Boolean constraints encoding

$$a_{\geq i} \Rightarrow a_{\geq j} \quad \text{and} \quad a_{\geq h} \Rightarrow a_{\geq i}$$

for the closest values $j < i < h$ that already exist

Integer Variables in Constraint Programming (2/2)

We can mix binary and order encodings! Define big-M linear inequalities encoding

$$a_{\geq 4} \Leftrightarrow -16a_{\text{neg}} + 1a_{b0} + 2a_{b1} + 4a_{b2} + 8a_{b3} \geq 4$$

$$a_{\geq 5} \Leftrightarrow -16a_{\text{neg}} + 1a_{b0} + 2a_{b1} + 4a_{b2} + 8a_{b3} \geq 5$$

$$a_{=4} \Leftrightarrow a_{\geq 4} \wedge \bar{a}_{\geq 5}$$

When creating $a_{\geq i}$, also introduce pseudo-Boolean constraints encoding

$$a_{\geq i} \Rightarrow a_{\geq j} \quad \text{and} \quad a_{\geq h} \Rightarrow a_{\geq i}$$

for the closest values $j < i < h$ that already exist

We can do this:

- Inside the pseudo-Boolean model where needed
- Otherwise lazily during proof logging

Compiling Constraints

- Also need to compile every constraint to pseudo-Boolean form
- Doesn't need to be a propagating encoding
- Can use additional variables

Compiling Linear Inequalities

Given inequality

$$2A + 3B + 4C \geq 42$$

where $A, B, C \in \{-3, \dots, 9\}$

Encode in pseudo-Boolean form as

$$\begin{aligned}
& -32a_{\text{neg}} + 2a_{b0} + 4a_{b1} + 8a_{b2} + 16a_{b3} \\
& + -48b_{\text{neg}} + 3b_{b0} + 6b_{b1} + 12b_{b2} + 24b_{b3} \\
& + -64c_{\text{neg}} + 4c_{b0} + 8c_{b1} + 16c_{b2} + 32c_{b3} \geq 42
\end{aligned}$$

Compiling Table Constraints

Constraints can be specified **extensionally** as list of feasible tuples, called a **table**
Variable assignments must match some row in table

Compiling Table Constraints

Constraints can be specified **extensionally** as list of feasible tuples, called a **table**
Variable assignments must match some row in table

Given table constraint

$$(A, B, C) \in [(1, 2, 3), (1, 3, 4), (2, 2, 5)]$$

define

$$3\bar{t}_1 + a_{=1} + b_{=2} + c_{=3} \geq 3$$

$$\text{i.e., } t_1 \Rightarrow (a_{=1} \wedge b_{=2} \wedge c_{=3})$$

$$3\bar{t}_2 + a_{=1} + b_{=3} + c_{=4} \geq 3$$

$$\text{i.e., } t_2 \Rightarrow (a_{=1} \wedge b_{=3} \wedge c_{=4})$$

$$3\bar{t}_3 + a_{=2} + b_{=2} + c_{=5} \geq 3$$

$$\text{i.e., } t_3 \Rightarrow (a_{=2} \wedge b_{=2} \wedge c_{=5})$$

using newly introduced tuple selector variables

$$t_1 + t_2 + t_3 = 1$$

Compiling Table Constraints

Constraints can be specified **extensionally** as list of feasible tuples, called a **table**
Variable assignments must match some row in table

Given table constraint

$$(A, B, C) \in [(1, 2, 3), (1, 3, 4), (2, 2, 5)]$$

define

$$3\bar{t}_1 + a_{=1} + b_{=2} + c_{=3} \geq 3$$

$$\text{i.e., } t_1 \Rightarrow (a_{=1} \wedge b_{=2} \wedge c_{=3})$$

$$3\bar{t}_2 + a_{=1} + b_{=3} + c_{=4} \geq 3$$

$$\text{i.e., } t_2 \Rightarrow (a_{=1} \wedge b_{=3} \wedge c_{=4})$$

$$3\bar{t}_3 + a_{=2} + b_{=2} + c_{=5} \geq 3$$

$$\text{i.e., } t_3 \Rightarrow (a_{=2} \wedge b_{=2} \wedge c_{=5})$$

using newly introduced tuple selector variables

$$t_1 + t_2 + t_3 = 1$$

Tuple variables **used for proof logging** — need not be known by solver (except for printing proofs)

Encoding Constraint Definitions

Already know how to do it for any constraint with a sane encoding using some combination of

- CNF
- Integer linear inequalities
- Table constraints
- Auxiliary variables

Simplicity is important, propagation strength isn't

Justifying Search

- Mostly this works out of the box
- Restarts are easy
- No need to prove anything about guesses/decisions — only justify backtracking

Justifying Inference

Key idea

Anything the constraint programming solver knows must follow from **unit propagation** of guessed assignments on **constraints in proof log**

Justifying Inference

Key idea

Anything the constraint programming solver knows must follow from **unit propagation** of guessed assignments on **constraints in proof log**

- If it follows from unit propagation on the encoding, nothing needed
- Some propagators and encodings need RUP steps for inferences
 - A lot of propagators are effectively “doing a little bit of lookahead” but in an efficient way

Justifying Inference

Key idea

Anything the constraint programming solver knows must follow from **unit propagation** of guessed assignments on **constraints in proof log**

If it follows from unit propagation on the encoding, nothing needed

Some propagators and encodings need RUP steps for inferences

- A lot of propagators are effectively “doing a little bit of lookahead” but in an efficient way

A few need explicit cutting planes justifications written to the proof log

- **Linear inequalities** just need to multiply and add
- **All-different** needs a bit more

Justifying All-Different Failures

$$\begin{aligned}
 V &\in \{ 1 \quad \quad \quad 4 \quad 5 \} \\
 W &\in \{ 1 \quad 2 \quad 3 \quad \quad \} \\
 X &\in \{ \quad 2 \quad 3 \quad \quad \} \\
 Y &\in \{ 1 \quad \quad 3 \quad \quad \} \\
 Z &\in \{ 1 \quad \quad 3 \quad \quad \}
 \end{aligned}$$

$$Z \in \{1, 3\}$$

Justifying All-Different Failures

$$V \in \{1 \quad \quad \quad 4 \quad 5\}$$

$$W \in \{1 \quad 2 \quad 3 \quad \quad \quad \} \quad w_{=1} + \quad w_{=2} + \quad w_{=3} \quad \geq \quad 1 \quad [\text{ } W \text{ takes some value}]$$

$$X \in \{ \quad 2 \quad 3 \quad \quad \quad \} \quad \quad \quad x_{=2} + \quad x_{=3} \quad \geq \quad 1 \quad [\text{ } X \text{ takes some value}]$$

$$Y \in \{1 \quad \quad \quad 3 \quad \quad \quad \} \quad y_{=1} \quad \quad \quad + \quad y_{=3} \quad \geq \quad 1 \quad [\text{ } Y \text{ takes some value}]$$

$$Z \in \{1 \quad \quad \quad 3 \quad \quad \quad \} \quad z_{=1} \quad \quad \quad + \quad z_{=3} \quad \geq \quad 1 \quad [\text{ } Z \text{ takes some value}]$$

Justifying All-Different Failures

$$V \in \{1 \quad \quad \quad 4 \quad 5\}$$

$$W \in \{1 \quad 2 \quad 3 \quad \quad \quad \} \quad w_{=1} + \quad w_{=2} + \quad w_{=3} \quad \quad \quad \geq \quad 1 \quad [\text{ } W \text{ takes some value}]$$

$$X \in \{ \quad 2 \quad 3 \quad \quad \quad \} \quad \quad \quad x_{=2} + \quad x_{=3} \quad \quad \quad \geq \quad 1 \quad [\text{ } X \text{ takes some value}]$$

$$Y \in \{1 \quad \quad \quad 3 \quad \quad \quad \} \quad y_{=1} \quad \quad \quad + \quad y_{=3} \quad \quad \quad \geq \quad 1 \quad [\text{ } Y \text{ takes some value}]$$

$$Z \in \{1 \quad \quad \quad 3 \quad \quad \quad \} \quad z_{=1} \quad \quad \quad + \quad z_{=3} \quad \quad \quad \geq \quad 1 \quad [\text{ } Z \text{ takes some value}]$$

$$\rightarrow \quad \quad \quad -v_{=1} + -w_{=1} + \quad \quad \quad -y_{=1} + -z_{=1} \geq -1 \quad [\text{ } \text{At most one variable} = 1 \text{ }]$$

$$\rightarrow \quad \quad \quad \quad \quad \quad -w_{=2} + -x_{=2} \quad \quad \quad \geq -1 \quad [\text{ } \text{At most one variable} = 2 \text{ }]$$

$$\rightarrow \quad \quad \quad \quad \quad \quad -w_{=3} + -x_{=3} + -y_{=3} + -z_{=3} \geq -1 \quad [\text{ } \text{At most one variable} = 3 \text{ }]$$

Justifying All-Different Failures

$$V \in \{1 \quad 4 \quad 5\}$$

$$W \in \{1 \quad 2 \quad 3 \quad \quad \quad \} \quad w_{=1} + w_{=2} + w_{=3} \geq 1 \quad [W \text{ takes some value}]$$

$$X \in \{ \quad 2 \quad 3 \quad \quad \quad \} \quad x_{=2} + x_{=3} \geq 1 \quad [X \text{ takes some value}]$$

$$Y \in \{1 \quad 3 \quad \quad \quad \} \quad y_{=1} + y_{=3} \geq 1 \quad [Y \text{ takes some value}]$$

$$Z \in \{1 \quad 3 \quad \quad \quad \} \quad z_{=1} + z_{=3} \geq 1 \quad [Z \text{ takes some value}]$$

$$\rightarrow \quad -v_{=1} + -w_{=1} + \quad -y_{=1} + -z_{=1} \geq -1 \quad [\text{At most one variable} = 1]$$

$$\rightarrow \quad \quad -w_{=2} + -x_{=2} \geq -1 \quad [\text{At most one variable} = 2]$$

$$\rightarrow \quad \quad -w_{=3} + -x_{=3} + -y_{=3} + -z_{=3} \geq -1 \quad [\text{At most one variable} = 3]$$

$$-v_{=1} \geq 1 \quad [\text{Sum all constraints so far}]$$

Justifying All-Different Failures

$$V \in \{1 \quad 4 \quad 5\}$$

$$W \in \{1 \quad 2 \quad 3 \quad \quad \quad \} \quad w_{=1} + w_{=2} + w_{=3} \geq 1 \quad [W \text{ takes some value}]$$

$$X \in \{ \quad 2 \quad 3 \quad \quad \quad \} \quad x_{=2} + x_{=3} \geq 1 \quad [X \text{ takes some value}]$$

$$Y \in \{1 \quad 3 \quad \quad \quad \} \quad y_{=1} + y_{=3} \geq 1 \quad [Y \text{ takes some value}]$$

$$Z \in \{1 \quad 3 \quad \quad \quad \} \quad z_{=1} + z_{=3} \geq 1 \quad [Z \text{ takes some value}]$$

$$\rightarrow \quad -v_{=1} + -w_{=1} + \quad -y_{=1} + -z_{=1} \geq -1 \quad [\text{At most one variable} = 1]$$

$$\rightarrow \quad \quad -w_{=2} + -x_{=2} \geq -1 \quad [\text{At most one variable} = 2]$$

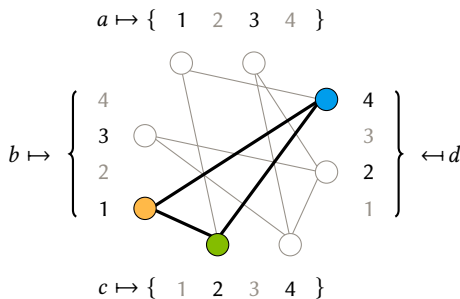
$$\rightarrow \quad \quad -w_{=3} + -x_{=3} + -y_{=3} + -z_{=3} \geq -1 \quad [\text{At most one variable} = 3]$$

$$-v_{=1} \geq 1 \quad [\text{Sum all constraints so far}]$$

$$v_{=1} \geq 0 \quad [\text{Variable } v_{=1} \text{ non-negative}]$$

$$V \in \{1, \dots, 4, 5\}$$

$$\begin{array}{llll}
\rightarrow & -v_{=1} + -w_{=1} + & -y_{=1} + -z_{=1} \geq -1 & \text{[At most one variable = 1]} \\
\rightarrow & & -w_{=2} + -x_{=2} & \geq -1 \quad \text{[At most one variable = 2]} \\
\rightarrow & & -w_{=3} + -x_{=3} + -y_{=3} + -z_{=3} \geq -1 & \text{[At most one variable = 3]} \\
\\
& -v_{=1} & \geq 1 & \text{[Sum all constraints so far]} \\
& v_{=1} & \geq 0 & \text{[Variable } v_{=1} \text{ non-negative]} \\
\\
& 0 & \geq 1 & \text{[Sum above two constraints]}
\end{array}$$



High-Level Modelling Languages?

High-level modelling languages like `MINIZINC` and `ESSENCE` have complicated compilers

How do we know we're giving a proof for the problem the user actually specified?

High-Level Modelling Languages?

High-level modelling languages like `MINIZINC` and `ESSENCE` have complicated compilers

How do we know we're giving a proof for the problem the user actually specified?

We don't

Need modelling language with formally specified semantics...

The *Glasgow Constraint Solver*

Support for **37 (families of) constraints** (and counting), with **all inferences certified**:

Abs, AllDifferent, AllEqual, Among, AtMostOne, BinPacking, Circuit, Comparison, Count, Cumulative, Diffn, Disjunctive, DivMod, Element, Equals, GCC, In, Increasing, Inverse, Knapsack, Lex, Linear, Logical, MDD, MinDistance, MinMax, Multiply, NValue, Parity, PlusMinus, Power, Regular, SeqPrecedeChain, SmartTable, Sort, Table, ValuePrecede

The *Glasgow Constraint Solver*

Support for **37 (families of) constraints** (and counting), with **all inferences certified**:

Abs, AllDifferent, AllEqual, Among, AtMostOne, BinPacking, Circuit, Comparison, Count, Cumulative, Diffn, Disjunctive, DivMod, Element, Equals, GCC, In, Increasing, Inverse, Knapsack, Lex, Linear, Logical, MDD, MinDistance, MinMax, Multiply, NValue, Parity, PlusMinus, Power, Regular, SeqPrecedeChain, SmartTable, Sort, Table, ValuePrecede

Also **certifies reformulations** not written in but inferred from input:

The *Glasgow Constraint Solver*

Support for **37 (families of) constraints** (and counting), with **all inferences certified**:

Abs, AllDifferent, AllEqual, Among, AtMostOne, BinPacking, Circuit, Comparison, Count, Cumulative, Diffn, Disjunctive, DivMod, Element, Equals, GCC, In, Increasing, Inverse, Knapsack, Lex, Linear, Logical, MDD, MinDistance, MinMax, Multiply, NValue, Parity, PlusMinus, Power, Regular, SeqPrecedeChain, SmartTable, Sort, Table, ValuePrecede

Also **certifies reformulations** not written in but inferred from input:

- Auto-tabulation: Identify small subproblem, solve it, and replace by table

The *Glasgow Constraint Solver*

Support for **37 (families of) constraints** (and counting), with **all inferences certified**:

Abs, AllDifferent, AllEqual, Among, AtMostOne, BinPacking, Circuit, Comparison, Count, Cumulative, Diffn, Disjunctive, DivMod, Element, Equals, GCC, In, Increasing, Inverse, Knapsack, Lex, Linear, Logical, MDD, MinDistance, MinMax, Multiply, NValue, Parity, PlusMinus, Power, Regular, SeqPrecedeChain, SmartTable, Sort, Table, ValuePrecede

Also **certifies reformulations** not written in but inferred from input:

- Auto-tabulation: Identify small subproblem, solve it, and replace by table
- Capacity strengthening by integrality, conflict cliques across resources, lifted cover cuts:
Extract information implied by scheduling problem constraints

The *Glasgow Constraint Solver*

Support for **37 (families of) constraints** (and counting), with **all inferences certified**:

Abs, AllDifferent, AllEqual, Among, AtMostOne, BinPacking, Circuit, Comparison, Count, Cumulative, Diffn, Disjunctive, DivMod, Element, Equals, GCC, In, Increasing, Inverse, Knapsack, Lex, Linear, Logical, MDD, MinDistance, MinMax, Multiply, NValue, Parity, PlusMinus, Power, Regular, SeqPrecedeChain, SmartTable, Sort, Table, ValuePrecede

Also **certifies reformulations** not written in but inferred from input:

- Auto-tabulation: Identify small subproblem, solve it, and replace by table
- Capacity strengthening by integrality, conflict cliques across resources, lifted cover cuts:
Extract information implied by scheduling problem constraints
- Difference logic: Spot that a pile of two-term inequalities is a precedence network, and hand to temporal propagator

The *Glasgow* Constraint Solver

Support for 37 (families of) constraints (and counting), with all inferences certified:

Abs, AllDifferent, AllEqual, Among, AtMostOne, BinPacking, Circuit, Comparison, Count, Cumulative, Diffn, Disjunctive, DivMod, Element, Equals, GCC, In, Increasing, Inverse, Knapsack, Lex, Linear, Logical, MDD, MinDistance, MinMax, Multiply, NValue, Parity, PlusMinus, Power, Regular, SeqPrecedeChain, SmartTable, Sort, Table, ValuePrecede

Also **certifies reformulations** not written in but inferred from input:

- Auto-tabulation: Identify small subproblem, solve it, and replace by table
- Capacity strengthening by integrality, conflict cliques across resources, lifted cover cuts: Extract information implied by scheduling problem constraints
- Difference logic: Spot that a pile of two-term inequalities is a precedence network, and hand to temporal propagator

Details in [EGMN20, GMN22, MM23, MMN24, MM25]

Code at <https://github.com/ciaranm/glasgow-constraint-solver>

Future Research Directions

Performance of pseudo-Boolean proof logging and checking

- Faster proof logging and checking!
- Trim proofs while checking as in SAT solving [HHW13] (*very recent work [ABB⁺26a]*)
- Compress proof files using binary format

Future Research Directions

Performance of pseudo-Boolean proof logging and checking

- Faster proof logging and checking!
- Trim proofs while checking as in SAT solving [HHW13] (*very recent work [ABB⁺26a]*)
- Compress proof files using binary format

Proof logging for other combinatorial problems and techniques

- Model counting
- Mixed integer linear programming (*suggested extension of VERIPB in [DEGH23]*)
- Satisfiability modulo theories (SMT) solving (*lots of work in, e.g., [HS22, BBC⁺23, RSB⁺26, RSS⁺26]*)

Future Research Directions

Performance of pseudo-Boolean proof logging and checking

- Faster proof logging and checking!
- Trim proofs while checking as in SAT solving [HHW13] (*very recent work [ABB⁺26a]*)
- Compress proof files using binary format

Proof logging for other combinatorial problems and techniques

- Model counting
- Mixed integer linear programming (*suggested extension of VERIPB in [DEGH23]*)
- Satisfiability modulo theories (SMT) solving (*lots of work in, e.g., [HS22, BBC⁺23, RSB⁺26, RSS⁺26]*)

And more...

- Use proof logs for algorithm analysis or explainability purposes
- Lots of other challenging problems and interesting ideas

Performance of pseudo-Boolean proof logging and checking

- ## Proof logging for other combinatorial problems and techniques

- ## And more...

- We're happy to collaborate, and we're hiring

Reminder About VeriPB Resources



VeriPB [tutorials, technical documentation, and more information](#) at
<https://VeriPB.org>

Tutorial series with [videos and slides](#) from *WHOOPS* '25 workshop at
<https://jakobnordstrom.se/WHOOPS25/>

[Technical details on different proof logging techniques](#) covered in research papers
[EGMN20, GMN20, GMM⁺20, GN21, GMN22, GMNO22, VDB22, BBN⁺23, BGMN23, MM23,
BBN⁺24, DMM⁺24, GMM⁺24, HOGN24, IOT⁺24, MMN24, DHN⁺25, JBBJ25, KLM⁺25, MM25,
ABB⁺26a, ABB⁺26b, DKK⁺26, MNOT26, VCB26] and PhD theses [Goc22, Mcl26, Oer26]

Lots of [concrete example files](#) at <https://gitlab.com/MIAOresearch/software/VeriPB>

Summing up

Second lecture

- More technical details on VERIPB
- Some convenient syntactic sugar (labels and reification)
- Example applications
 - 1 Maximum clique
 - 2 Subgraph isomorphism
 - 3 Constraint programming

Main message

- Combinatorial solvers are amazing, but **ensuring correctness is crucial**
- **Certifying solvers** producing **machine-verifiable proofs** is the most promising approach
- **Pseudo-Boolean proof logging** has been surprisingly successful in moving the research frontier forward

Summing up

Second lecture

- More technical details on VERIPB
- Some convenient syntactic sugar (labels and reification)
- Example applications
 - 1 Maximum clique
 - 2 Subgraph isomorphism
 - 3 Constraint programming

Main message

- Combinatorial solvers are amazing, but **ensuring correctness is crucial**
- **Certifying solvers** producing **machine-verifiable proofs** is the most promising approach
- **Pseudo-Boolean proof logging** has been surprisingly successful in moving the research frontier forward
- **Action point:** What problems can VERIPB solve for you? 😊

References I

- [ABB⁺26a] Berhan Oumer Adame, Bart Bogaerts, Benjamin Bogø, Simon Dold, Arthur Gontier, Wietze Koops, Ciaran McCreesh, Matthew McIlree, Jakob Nordström, Andy Oertel, Adrián Rebola-Pardo, and Mark Turnbull. Trimming pseudo-Boolean proofs. In *Proceedings of the 26th Conference on Formal Methods in Computer-Aided Design (FMCAD '26)*, pages 743–756, September 2026.
- [ABB⁺26b] Markus Anders, Bart Bogaerts, Benjamin Bogø, Arthur Gontier, Wietze Koops, Ciaran McCreesh, Magnus O. Myreen, Jakob Nordström, Andy Oertel, Adrián Rebola-Pardo, and Yong Kiam Tan. Faster certified symmetry breaking using orders with auxiliary variables. In *Proceedings of the 40th AAAI Conference on Artificial Intelligence (AAAI '26)*, pages 14140–14148, January 2026.
- [Bar95] Peter Barth. A Davis-Putnam based enumeration algorithm for linear pseudo-Boolean optimization. Technical Report MPI-I-95-2-003, Max-Planck-Institut für Informatik, January 1995.
- [BBC⁺23] Haniel Barbosa, Clark Barrett, Byron Cook, Bruno Dutertre, Gereon Kremer, Hanna Lachnitt, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Cesare Tinelli, and Yoni Zohar. Generating and exploiting automated reasoning proof certificates. *Communications of the ACM*, 66(10):86–95, October 2023.
- [BBN⁺23] Jeremias Berg, Bart Bogaerts, Jakob Nordström, Andy Oertel, and Dieter Vandesande. Certified core-guided MaxSAT solving. In *Proceedings of the 29th International Conference on Automated Deduction (CADE-29)*, volume 14132 of *Lecture Notes in Computer Science*, pages 1–22. Springer, July 2023.
- [BBN⁺24] Jeremias Berg, Bart Bogaerts, Jakob Nordström, Andy Oertel, Tobias Paxian, and Dieter Vandesande. Certifying without loss of generality reasoning in solution-improving maximum satisfiability. In *Proceedings of the 30th International Conference on Principles and Practice of Constraint Programming (CP '24)*, volume 307 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 4:1–4:28, September 2024.

References II

- [BGMN23] Bart Bogaerts, Stephan Gocht, Ciaran McCreesh, and Jakob Nordström. Certified dominance and symmetry breaking for combinatorial optimisation. *Journal of Artificial Intelligence Research*, 77:1539–1589, August 2023. Preliminary version in *AAAI '22*.
- [CCT87] William Cook, Collette Rene Coullard, and György Turán. On the complexity of cutting-plane proofs. *Discrete Applied Mathematics*, 18(1):25–38, November 1987.
- [DEGH23] Jasper van Doornmalen, Leon Eifler, Ambros Gleixner, and Christopher Hojny. A proof system for certifying symmetry and optimality reasoning in integer programming. Technical Report 2311.03877, arXiv.org, November 2023.
- [DHN⁺25] Simon Dold, Malte Helmert, Jakob Nordström, Gabriele Röger, and Tanja Schindler. Pseudo-Boolean proof logging for optimal classical planning. In *Proceedings of the 35th International Conference on Automated Planning and Scheduling (ICAPS '25)*, pages 54–63, November 2025.
- [DKK⁺26] Simon Dold, George Katsirelos, Wietze Koops, Magnus O. Myreen, Jakob Nordström, Andy Oertel, and Yong Kiam Tan. End-to-end certified graph colouring. In *Proceedings of the 32nd International Conference on Principles and Practice of Constraint Programming (CP '26)*, volume 379 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 21:1–21:27, July 2026.
- [DMM⁺24] Emir Demirović, Ciaran McCreesh, Matthew McIlree, Jakob Nordström, Andy Oertel, and Konstantin Sidorov. Pseudo-Boolean reasoning about states and transitions to certify dynamic programming and decision diagram algorithms. In *Proceedings of the 30th International Conference on Principles and Practice of Constraint Programming (CP '24)*, volume 307 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 9:1–9:21, September 2024.
- [EGMN20] Jan Elffers, Stephan Gocht, Ciaran McCreesh, and Jakob Nordström. Justifying all differences using pseudo-Boolean reasoning. In *Proceedings of the 34th AAAI Conference on Artificial Intelligence (AAAI '20)*, pages 1486–1494, February 2020.

References III

- [GMM⁺20] Stephan Gocht, Ross McBride, Ciaran McCreesh, Jakob Nordström, Patrick Prosser, and James Trimble. Certifying solvers for clique and maximum common (connected) subgraph problems. In *Proceedings of the 26th International Conference on Principles and Practice of Constraint Programming (CP '20)*, volume 12333 of *Lecture Notes in Computer Science*, pages 338–357. Springer, September 2020.
- [GMM⁺24] Stephan Gocht, Ciaran McCreesh, Magnus O. Myreen, Jakob Nordström, Andy Oertel, and Yong Kiam Tan. End-to-end verification for subgraph solving. In *Proceedings of the 38th AAAI Conference on Artificial Intelligence (AAAI '24)*, pages 8038–8047, February 2024.
- [GMN20] Stephan Gocht, Ciaran McCreesh, and Jakob Nordström. Subgraph isomorphism meets cutting planes: Solving with certified solutions. In *Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI '20)*, pages 1134–1140, July 2020.
- [GMN22] Stephan Gocht, Ciaran McCreesh, and Jakob Nordström. An auditable constraint programming solver. In *Proceedings of the 28th International Conference on Principles and Practice of Constraint Programming (CP '22)*, volume 235 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 25:1–25:18, August 2022.
- [GMNO22] Stephan Gocht, Ruben Martins, Jakob Nordström, and Andy Oertel. Certified CNF translations for pseudo-Boolean solving. In *Proceedings of the 25th International Conference on Theory and Applications of Satisfiability Testing (SAT '22)*, volume 236 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 16:1–16:25, August 2022.
- [GN21] Stephan Gocht and Jakob Nordström. Certifying parity reasoning efficiently using pseudo-Boolean proofs. In *Proceedings of the 35th AAAI Conference on Artificial Intelligence (AAAI '21)*, pages 3768–3777, February 2021.
- [Goc22] Stephan Gocht. *Certifying Correctness for Combinatorial Algorithms by Using Pseudo-Boolean Reasoning*. PhD thesis, Lund University, June 2022. Available at <https://portal.research.lu.se/en/publications/certifying-correctness-for-combinatorial-algorithms-by-using-pseu>.

References IV

- [HHW13] Marijn J. H. Heule, Warren A. Hunt Jr., and Nathan Wetzler. Trimming while checking clausal proofs. In *Proceedings of the 13th International Conference on Formal Methods in Computer-Aided Design (FMCAD '13)*, pages 181–188, October 2013.
- [HOGN24] Alexander Hoen, Andy Oertel, Ambros Gleixner, and Jakob Nordström. Certifying MIP-based presolve reductions for 0–1 integer linear programs. In *Proceedings of the 21st International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR '24)*, volume 14742 of *Lecture Notes in Computer Science*, pages 310–328. Springer, May 2024.
- [HS22] Jochen Hoenicke and Tanja Schindler. A simple proof format for SMT. In *Proceedings of the 20th International Workshop on Satisfiability Modulo Theories (SMT '22)*, volume 3185 of *CEUR Workshop Proceedings*, pages 54–70, August 2022.
- [IOT⁺24] Hannes Ihalainen, Andy Oertel, Yong Kiam Tan, Jeremias Berg, Matti Järvisalo, Magnus O. Myreen, and Jakob Nordström. Certified MaxSAT preprocessing. In *Proceedings of the 12th International Joint Conference on Automated Reasoning (IJCAR '24)*, volume 14739 of *Lecture Notes in Computer Science*, pages 396–418. Springer, July 2024.
- [JBBJ25] Christoph Jabs, Jeremias Berg, Bart Bogaerts, and Matti Järvisalo. Certifying Pareto-optimality in multi-objective maximum satisfiability. In *Proceedings of the 31st International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS '25)*, volume 15697 of *Lecture Notes in Computer Science*, pages 108–129. Springer, May 2025.
- [JHB12] Matti Järvisalo, Marijn J. H. Heule, and Armin Biere. Inprocessing rules. In *Proceedings of the 6th International Joint Conference on Automated Reasoning (IJCAR '12)*, volume 7364 of *Lecture Notes in Computer Science*, pages 355–370. Springer, June 2012.
- [KLM⁺25] Wietze Koops, Daniel Le Berre, Magnus O. Myreen, Jakob Nordström, Andy Oertel, Yong Kiam Tan, and Marc Vinyals. Practically feasible proof logging for pseudo-Boolean optimization. In *Proceedings of the 31st International Conference on Principles and Practice of Constraint Programming (CP '25)*, volume 340 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 21:1–21:27, August 2025.

References V

- [McI26] Matthew McIlree. *Pseudo-Boolean Proof Logging for Constraint Propagation Algorithms*. PhD thesis, University of Glasgow, June 2026. Available at <https://theses.gla.ac.uk/86049/>.
- [MM23] Matthew McIlree and Ciaran McCreesh. Proof logging for smart extensional constraints. In *Proceedings of the 29th International Conference on Principles and Practice of Constraint Programming (CP '23)*, volume 280 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 26:1–26:17, August 2023.
- [MM25] Matthew McIlree and Ciaran McCreesh. Certifying bounds propagation for integer multiplication constraints. In *Proceedings of the 39th AAAI Conference on Artificial Intelligence (AAAI '25)*, pages 11309–11317, February–March 2025.
- [MMN24] Matthew McIlree, Ciaran McCreesh, and Jakob Nordström. Proof logging for the circuit constraint. In *Proceedings of the 21st International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR '24)*, volume 14743 of *Lecture Notes in Computer Science*, pages 38–55. Springer, May 2024.
- [MNOT26] Ciaran McCreesh, Jakob Nordström, Andy Oertel, and Yong Kiam Tan. Proof logging for projected enumeration (and counting?) problems in VeriPB. In *Proceedings of the 32nd International Conference on Principles and Practice of Constraint Programming (CP '26)*, volume 379 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 43:1–43:21, July 2026.
- [MP16] Ciaran McCreesh and Patrick Prosser. Finding maximum k -cliques faster using lazy global domination. In *Proceedings of the 9th Annual Symposium on Combinatorial Search (SOCS '16)*, pages 72–80, July 2016.
- [MPP19] Ciaran McCreesh, William Pettersson, and Patrick Prosser. Understanding the empirical hardness of random optimisation problems. In *Proceedings of the 25th International Conference on Principles and Practice of Constraint Programming (CP '19)*, volume 11802 of *Lecture Notes in Computer Science*, pages 333–349. Springer, September–October 2019.

References VI

- [Oer26] Andy Oertel. *Certifying Combinatorial Optimization: A Unified Approach Using Pseudo-Boolean Reasoning*. PhD thesis, Lund University, May 2026. Available at <https://portal.research.lu.se/sv/publications/certifying-combinatorial-optimization-a-unified-approach-using-ps>.
- [RSB⁺26] Andrew Reynolds, Hans-Jörg Schurr, Haniel Barbosa, Ofec Israel, Jibiana Jakpor, Hanna Lachnitt, Abdalrhman Mohamed, Aina Niemetz, Mathias Preiner, Yoni Zohar, Robert B. Jones, Clark W. Barrett, and Cesare Tinelli. The cooperating proof calculus: Comprehensive proofs for an SMT solver. In *Proceedings of the 38th International Conference on Computer Aided Verification (CAV '26), Part II*, volume 16683 of *Lecture Notes in Computer Science*, pages 188–212. Springer, July 2026.
- [RSS⁺26] Andrew Reynolds, Hans-Jörg Schurr, Mallku Soldevila, Haniel Barbosa, Clark W. Barrett, and Cesare Tinelli. Ethos: A fast proof checker for the Eunoia logical framework. In *Proceedings of the 13th International Joint Conference on Automated Reasoning (IJCAR '26), Part I*, volume 16688 of *Lecture Notes in Computer Science*, pages 305–314. Springer, July 2026.
- [VCB26] Dieter Vandesande, Jordi Coll, and Bart Bogaerts. Certified branch-and-bound MaxSAT solving. In *Proceedings of the 40th AAAI Conference on Artificial Intelligence (AAAI '26)*, pages 14342–14351, January 2026.
- [VDB22] Dieter Vandesande, Wolf De Wulf, and Bart Bogaerts. QMaxSATpb: A certified MaxSAT solver. In *Proceedings of the 16th International Conference on Logic Programming and Non-monotonic Reasoning (LPNMR '22)*, volume 13416 of *Lecture Notes in Computer Science*, pages 429–442. Springer, September 2022.